

# ANT-X SIMULATION MATLAB

*PRIMA VERSIONE DELLA SIMULAZIONE MATLAB PER IL DRONE UAV ANT-X*

---

MATTEO TAVANO

*MSc Student in Data Science and Artificial Intelligence*

*Università degli studi di Trieste*

*[GitHub](#)*

`matteo.tavano@studenti.units.it`

---

# Contents

|            |                                             |           |
|------------|---------------------------------------------|-----------|
| <b>I</b>   | <b>Introduzione</b>                         | <b>1</b>  |
| 1          | Utilizzo e Prerequisiti                     | 1         |
| 2          | Limitazioni Principali                      | 2         |
| <b>II</b>  | <b>Modellazione del Sistema</b>             | <b>2</b>  |
| <b>3</b>   | <b>Modello Dinamico del Quadcopter</b>      | <b>2</b>  |
| 3.1        | Derivazione del Modello                     | 4         |
| 3.1.1      | Cinematica della Posizione                  | 4         |
| 3.1.2      | Dinamica Traslazionale                      | 5         |
| 3.1.3      | Cinematica Rotazionale                      | 6         |
| 3.1.4      | Dinamica Rotazionale                        | 7         |
| 3.2        | Modello Dinamico a 13 Stati                 | 8         |
| 3.2.1      | Cinematica della Posizione                  | 8         |
| 3.2.2      | Dinamica Traslazionale                      | 9         |
| 3.2.3      | Cinematica dell'Assetto tramite Quaternioni | 9         |
| 3.3        | Parametri                                   | 9         |
| 3.4        | Linearizzazione                             | 10        |
| 3.4.1      | Jacobiana rispetto agli Stati               | 10        |
| 3.4.2      | Jacobiana rispetto agli Input               | 10        |
| <b>4</b>   | <b>Flatness Map</b>                         | <b>11</b> |
| 4.1        | State Reconstruction                        | 12        |
| 4.2        | Input Reconstruction                        | 13        |
| <b>III</b> | <b>Definizione delle Traiettorie</b>        | <b>14</b> |
| <b>5</b>   | <b>Test sui Gradi di Libertà (DoF)</b>      | <b>14</b> |
| 5.1        | Test Longitudinale (Pitch)                  | 16        |
| 5.2        | Test Laterale (Roll)                        | 16        |
| 5.3        | Test Verticale (Thrust)                     | 16        |
| 5.4        | Test Direzionale (Yaw)                      | 17        |
| <b>6</b>   | <b>Traiettorie Complesse</b>                | <b>17</b> |
| 6.1        | Traiettoria Circolare                       | 17        |
| 6.2        | Lemniscata di Geronio                       | 18        |
| 6.3        | Traiettoria Spirale                         | 18        |

|           |                                                              |           |
|-----------|--------------------------------------------------------------|-----------|
| <b>7</b>  | <b>Open-Loop Test</b>                                        | <b>19</b> |
| 7.1       | Open-Loop Test — Cerchio                                     | 20        |
| 7.2       | Open-Loop Test — Lemniscata                                  | 21        |
| 7.3       | Open-Loop Test — Spirale                                     | 22        |
| 7.4       | Open-Loop: Riferimento Continuo vs Riferimento con Input PWC | 23        |
| <br>      |                                                              |           |
| <b>IV</b> | <b>Implementazione MPC</b>                                   | <b>24</b> |
| <br>      |                                                              |           |
| <b>8</b>  | <b>LTV-MPC</b>                                               | <b>24</b> |
| 8.1       | Parametri di Controllo                                       | 24        |
| 8.2       | Strategia di Ottimizzazione                                  | 25        |
| 8.3       | Costruzione del Problema QP                                  | 25        |
| 8.4       | Warm Start                                                   | 26        |
| <br>      |                                                              |           |
| <b>9</b>  | <b>Simulazione in Closed-Loop</b>                            | <b>26</b> |
| 9.1       | Traiettorie Testate                                          | 26        |
| 9.2       | Condizioni Iniziali                                          | 26        |
| 9.3       | Metriche di Valutazione                                      | 28        |
| 9.4       | Output della Simulazione                                     | 28        |
| <br>      |                                                              |           |
| <b>V</b>  | <b>Risultati e Conclusioni</b>                               | <b>29</b> |
| <br>      |                                                              |           |
| <b>10</b> | <b>Risultati</b>                                             | <b>29</b> |
| 10.1      | Tracking MPC                                                 | 30        |
| 10.2      | Tracking dello Stato                                         | 31        |
| 10.3      | Analisi del Tracking                                         | 32        |
| 10.4      | Tracking degli Input                                         | 33        |
| 10.5      | Risultati Quantitativi                                       | 34        |
| <br>      |                                                              |           |
| <b>11</b> | <b>Implementazioni Future e Conclusioni</b>                  | <b>35</b> |

---

# Introduzione

Questa repository ha l'obiettivo di raccogliere e documentare le prime versioni della simulazione MATLAB relative al drone UAV **ANT-X** [1]. Si tratta di una fase iniziale del progetto, finalizzata principalmente all'organizzazione delle idee e alla definizione di un modello semplificato in grado di rappresentare le principali dinamiche del drone ANT-X.

L'obiettivo principale è sviluppare, ad alto livello, l'intera **pipeline operativa** della simulazione, comprendente:

1. definizione del *modello dinamico* non lineare;
2. *ricostruzione* di stati e input a partire dalle *uscite piate*;
3. generazione di *traiettorie feasible*;
4. costruzione offline delle *reference* tramite input PWC (Piecewise Constant);
5. validazione iniziale in *open-loop*;
6. implementazione del controllore *MPC*;
7. raccolta e analisi dei risultati ottenuti dalle **simulazioni**.

La repository rappresenta quindi una **base di sviluppo** per l'evoluzione futura del framework di simulazione e controllo del drone **ANT-X**.

## SECTION 1

## Utilizzo e Prerequisiti

---

Per eseguire le simulazioni e ispezionare il codice è necessario disporre di una versione aggiornata di MATLAB con le librerie principali richieste dal progetto, in particolare:

- **ode45** per l'integrazione numerica del modello dinamico non-lineare;
- **Optimization Toolbox** per l'utilizzo del risolutore quadprog, impiegato nell'implementazione del controllore MPC.

Le simulazioni disponibili sono contenute nella directory `scripts/` e comprendono le seguenti categorie:

- **Simulazione Open-Loop di base:** Utilizzata per validare i principali gradi di libertà del drone, analizzando separatamente le dinamiche di Roll, Pitch, Thrust e Yaw.
- **Simulazioni Open-Loop su traiettorie complesse:** Finalizzate alla validazione di traiettorie generate tramite differential flatness, tra cui Circle, Lemniscate e Spiral, integrando il modello dinamico con input campionati.
- **Confronto tra traiettorie continue e input PWC:** Simulazione dedicata al confronto tra traiettorie continue ideali e traiettorie ottenute tramite integrazione di input Piecewise Constant (PWC).
- **Simulazioni Closed-Loop con MPC:** Test del controllore Model Predictive Control (MPC) su diverse condizioni iniziali e su differenti traiettorie di riferimento.

## Limitazioni Principali

---

- **Corpo rigido idealizzato:** I modelli implementati trascurano il drag aerodinamico, il blade flapping e le dinamiche dei rotori.
- **Assenza della dinamica motore/ESC:** Il modello assume attuazione istantanea.
- **Mixing non implementato:** Gli ingressi del controllore sono grandezze fisiche  $[T, L, M, N]$ ; i coefficienti di thrust  $C_T$  e di torque  $C_Q$  (necessari per la mappatura verso i 4 segnali PWM motore) non sono noti e la matrice di mixing non è inclusa.
- **Matrici di inerzia:** I valori  $J_{xx}$ ,  $J_{yy}$ ,  $J_{zz}$  sono ereditati da una stima su drone simile e dovranno essere validati o sostituiti con misure sperimentali in laboratorio. [2]
- **Saturazioni di input come stime iniziali:** I limiti  $T \in [0, 2mg \text{ N}]$ ,  $L, M \in [\pm 0.15 \text{ Nm}]$ ,  $N \in [\pm 0.05 \text{ Nm}]$  sono stime preliminari.
- **Vincoli sui quaternioni nel modello a 13 stati:** I vincoli sui quaternioni sono stati omessi nel MPC.
- **Matrici di peso  $Q$ ,  $R$  e costo terminale  $Q_f = Q$ :** Valori inizializzati e adattati da Kunz et al. [3]; la scelta  $Q_f = Q$  è una semplificazione ereditata dal paper che non garantisce stabilità asintotica.
- **quadprog non è real-time embeddable.** Per il deployment onboard il solver QP dovrà essere sostituito con un'alternativa real-time.
- **Stato perfetto (no EKF, no rumore).** La simulazione assume disponibilità completa e priva di rumore dello stato del sistema. In laboratorio, lo stato deriverà dalla fusione IMU + motion capture tramite l'EKF di PX4, con rumore di misura e latenza.
- **Ground effect e decollo non modellati.** La condizione iniziale `Origin` posiziona il drone a terra, ma l'aumento di spinta dovuto al **ground effect** e la traiettoria di decollo verso la quota nominale non sono gestiti esplicitamente.

# Modellazione del Sistema

## Modello Dinamico del Quadcopter

---

Il framework utilizza un modello dinamico non lineare semplificato del quadcopter, implementato nella classe `A00_SimplifiedModel.m`. Il modello descrive la dinamica rigida del drone tramite le equazioni di Newton-Eulero, assumendo il velivolo come corpo rigido e trascurando, in questa prima fase, effetti aerodinamici avanzati, dinamiche dei motori e disturbi esterni.

La formulazione adottata utilizza:

PART

II

- un sistema di riferimento inerziale **NED** (*North-East-Down*);
- un sistema di riferimento body **FRD** (*Forward-Right-Down*);
- velocità lineari espresse nel body frame;
- orientazione descritta tramite angoli di Eulero **Z-Y-X**.

Lo **stato** del sistema è definito da:

$$[x_I, y_I, z_I, \dot{x}_B, \dot{y}_B, \dot{z}_B, \phi, \theta, \psi, p, q, r] \in \mathbb{R}^{12}$$

dove:

- $p_I = (x_I, y_I, z_I)$  rappresentano le posizioni nel frame inerziale in [m];
- $v_B = (\dot{x}_B, \dot{y}_B, \dot{z}_B)$  sono le velocità lineari nel body frame in [m/s];
- $\alpha = (\phi, \theta, \psi)$  corrispondono agli angoli di assetto (Roll, Pitch, Yaw) in [rad];
- $\omega_B = (p, q, r)$  sono le velocità angolari del corpo rigido in [rad/s].

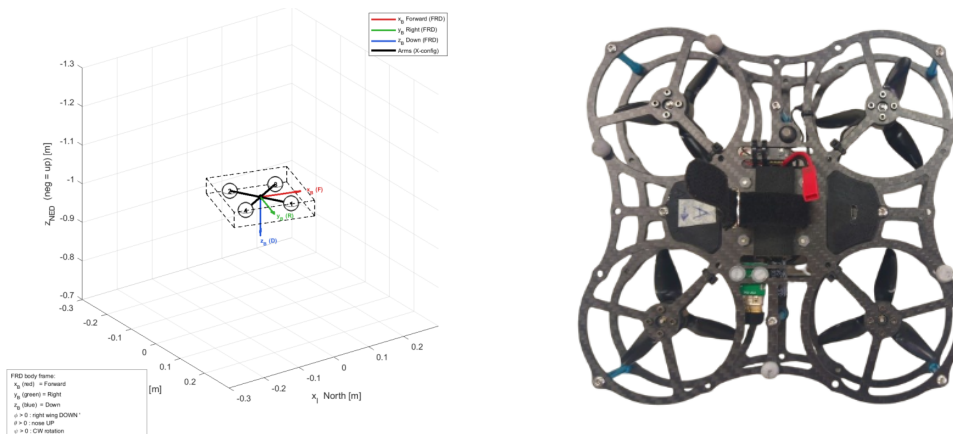
Gli **ingressi** del modello corrispondono invece a:

$$[T, L, M, N] \in \mathbb{R}^4$$

dove:

- $T$  è il thrust collettivo [N];
- $L$  il momento di Roll [Nm];
- $M$  il momento di Pitch [Nm];
- $N$  il momento di Yaw [Nm].

*Remark* In questa prima implementazione, gli ingressi del modello rappresentano direttamente grandezze fisiche non normalizzate. In particolare, il thrust collettivo agisce lungo l'asse  $z_B$  del body frame, mentre le coppie  $L$ ,  $M$  e  $N$  influenzano rispettivamente le dinamiche di Roll, Pitch e Yaw attraverso i body rates  $p$ ,  $q$  e  $r$ .



**Figure 1.** Confronto tra la ricostruzione del drone (sinistra) e il modello ANT-X (destra).

## SUBSECTION 3.1

**Derivazione del Modello**

La funzione `dynamics()` implementa il sistema di ODE composto da 4 blocchi: cinematica della posizione, dinamica traslazionale, cinematica rotazionale e dinamica rotazionale.

$$\left\{ \begin{array}{l} \dot{x}_I = \dot{x}_B(\cos \theta \cos \psi) + \dot{y}_B(\sin \phi \sin \theta \cos \psi - \cos \phi \sin \psi) + \dot{z}_B(\cos \phi \sin \theta \cos \psi + \sin \phi \sin \psi) \\ \dot{y}_I = \dot{x}_B(\cos \theta \sin \psi) + \dot{y}_B(\sin \phi \sin \theta \sin \psi + \cos \phi \cos \psi) + \dot{z}_B(\cos \phi \sin \theta \sin \psi - \sin \phi \cos \psi) \\ \dot{z}_I = -\dot{x}_B \sin \theta + \dot{y}_B \sin \phi \cos \theta + \dot{z}_B \cos \phi \cos \theta \\ \ddot{x}_B = r \dot{y}_B - q \dot{z}_B - g \sin \theta \\ \ddot{y}_B = p \dot{z}_B - r \dot{x}_B + g \sin \phi \cos \theta \\ \ddot{z}_B = q \dot{x}_B - p \dot{y}_B + g \cos \phi \cos \theta - \frac{T}{m} \\ \dot{\phi} = p + \sin \phi \tan \theta q + \cos \phi \tan \theta r \\ \dot{\theta} = \cos \phi q - \sin \phi r \\ \dot{\psi} = \frac{\sin \phi}{\cos \theta} q + \frac{\cos \phi}{\cos \theta} r \\ \dot{p} = \frac{1}{J_{xx}} (L - (J_{zz} - J_{yy})qr) \\ \dot{q} = \frac{1}{J_{yy}} (M - (J_{xx} - J_{zz})pr) \\ \dot{r} = \frac{1}{J_{zz}} (N - (J_{yy} - J_{xx})pq) \end{array} \right.$$

**3.1.1 Cinematica della Posizione**

Le velocità lineari del drone vengono espresse nel body frame:

$$v_B = \begin{bmatrix} \dot{x}_B \\ \dot{y}_B \\ \dot{z}_B \end{bmatrix}$$

mentre la posizione nel frame inerziale è definita come:

$$p_I = \begin{bmatrix} x_I \\ y_I \\ z_I \end{bmatrix}$$

Applicando la sequenza di rotazioni Z-Y-X, attraverso le matrici:

$$R_z(\psi) = \begin{bmatrix} \cos \psi & -\sin \psi & 0 \\ \sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix}$$

$$R_x(\phi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \phi & -\sin \phi \\ 0 & \sin \phi & \cos \phi \end{bmatrix}$$

otteniamo la seguente equivalenza:

$$R(\phi, \theta, \psi) = R_z(\psi)R_y(\theta)R_x(\phi)$$

$$R(\phi, \theta, \psi) = \begin{bmatrix} \cos \theta \cos \psi & \sin \phi \sin \theta \cos \psi - \cos \phi \sin \psi & \cos \phi \sin \theta \cos \psi + \sin \phi \sin \psi \\ \cos \theta \sin \psi & \sin \phi \sin \theta \sin \psi + \cos \phi \cos \psi & \cos \phi \sin \theta \sin \psi - \sin \phi \cos \psi \\ -\sin \theta & \sin \phi \cos \theta & \cos \phi \cos \theta \end{bmatrix}$$

La derivata della posizione nel frame inerziale è quindi:

$$\dot{p}_I = R(\phi, \theta, \psi) v_B$$

Esplicitando le tre equazioni si ottiene:

$$\begin{cases} \dot{x}_I = \dot{x}_B(\cos \theta \cos \psi) + \dot{y}_B(\sin \phi \sin \theta \cos \psi - \cos \phi \sin \psi) + \dot{z}_B(\cos \phi \sin \theta \cos \psi + \sin \phi \sin \psi) \\ \dot{y}_I = \dot{x}_B(\cos \theta \sin \psi) + \dot{y}_B(\sin \phi \sin \theta \sin \psi + \cos \phi \cos \psi) + \dot{z}_B(\cos \phi \sin \theta \sin \psi - \sin \phi \cos \psi) \\ \dot{z}_I = -\dot{x}_B \sin \theta + \dot{y}_B \sin \phi \cos \theta + \dot{z}_B \cos \phi \cos \theta \end{cases}$$

### 3.1.2 Dinamica Traslazionale

La dinamica traslazionale descrive come variano le velocità lineari del quadcopter espresse nel sistema di riferimento Body Frame (FRD):

$$v_B = \begin{bmatrix} \dot{x}_B \\ \dot{y}_B \\ \dot{z}_B \end{bmatrix}$$

Partendo dalla seconda legge di Newton:

$$m \mathbf{a}_I = \mathbf{F}_I$$

La derivata temporale della velocità inerziale è:

$$\mathbf{a}_I = \frac{d}{dt} \mathbf{v}_I = \frac{d}{dt} (\mathbf{R} \mathbf{v}_B) = \mathbf{R} \dot{\mathbf{v}}_B + \dot{\mathbf{R}} \mathbf{v}_B$$

Sfruttando la proprietà cinematica della derivata della matrice di rotazione  $\dot{\mathbf{R}} = \mathbf{R} \mathbf{S}(\boldsymbol{\omega}_B)$ , dove  $\mathbf{S}(\boldsymbol{\omega}_B)$  è la matrice antisimmetrica associata alla velocità angolare del corpo  $\boldsymbol{\omega}_B = [p, q, r]^T$ , si ottiene:

$$\mathbf{a}_I = \mathbf{R} (\dot{\mathbf{v}}_B + \boldsymbol{\omega}_B \times \mathbf{v}_B)$$

Sostituendo questa espressione nella legge di Newton e proiettando tutte le forze sul Body frame ( $\mathbf{F}_B = \mathbf{R}^T \mathbf{F}_I$ ), otteniamo l'equazione fondamentale della dinamica traslazionale:

$$m (\dot{\mathbf{v}}_B + \boldsymbol{\omega}_B \times \mathbf{v}_B) = \mathbf{F}_B$$

Sul quadcopter agiscono principalmente due forze esterne: la gravità e la spinta collettiva generata dai rotori.



- **Forza di Gravità** ( $\mathbf{F}_{g,B}$ ): la forza di gravità nel sistema inerziale (NED) è diretta esclusivamente lungo l'asse  $z_I$ :

$$\mathbf{F}_{g,I} = \begin{bmatrix} 0 \\ 0 \\ mg \end{bmatrix}$$

Proiettando questa forza nel Body frame tramite la trasposta della matrice di rotazione  $\mathbf{R}^T$ :

$$\mathbf{F}_{g,B} = \mathbf{R}^T \begin{bmatrix} 0 \\ 0 \\ mg \end{bmatrix} = mg \begin{bmatrix} -\sin \theta \\ \sin \phi \cos \theta \\ \cos \phi \cos \theta \end{bmatrix}$$

- **Spinta dei motori** ( $\mathbf{F}_{T,B}$ ): i motori generano una spinta collettiva totale  $T$  diretta lungo l'asse verticale del corpo, verso l'alto (direzione  $-z_B$  nel sistema di riferimento Down):

$$\mathbf{F}_{T,B} = \begin{bmatrix} 0 \\ 0 \\ -T \end{bmatrix}$$

- **Forza totale** ( $\mathbf{F}_B$ ): sommando i contributi della gravità e della spinta si ottiene:

$$\mathbf{F}_B = \mathbf{F}_{g,B} + \mathbf{F}_{T,B} = \begin{bmatrix} -mg \sin \theta \\ mg \sin \phi \cos \theta \\ mg \cos \phi \cos \theta - T \end{bmatrix}$$

Sviluppando il termine cinematico legato al prodotto vettoriale della velocità angolare  $\boldsymbol{\omega}_B = [p, q, r]^T$  e della velocità lineare  $\mathbf{v}_B = [\dot{x}_B, \dot{y}_B, \dot{z}_B]^T$ :

$$\boldsymbol{\omega}_B \times \mathbf{v}_B = \begin{bmatrix} p \\ q \\ r \end{bmatrix} \times \begin{bmatrix} \dot{x}_B \\ \dot{y}_B \\ \dot{z}_B \end{bmatrix} = \begin{bmatrix} q\dot{z}_B - r\dot{y}_B \\ r\dot{x}_B - p\dot{z}_B \\ p\dot{y}_B - q\dot{x}_B \end{bmatrix}$$

Sostituendo questa relazione nell'equazione di Newton-Euler:

$$m \left( \begin{bmatrix} \ddot{x}_B \\ \ddot{y}_B \\ \ddot{z}_B \end{bmatrix} + \begin{bmatrix} q\dot{z}_B - r\dot{y}_B \\ r\dot{x}_B - p\dot{z}_B \\ p\dot{y}_B - q\dot{x}_B \end{bmatrix} \right) = \begin{bmatrix} -mg \sin \theta \\ mg \sin \phi \cos \theta \\ mg \cos \phi \cos \theta - T \end{bmatrix}$$

Dividendo per la massa  $m$  e isolando le accelerazioni  $\ddot{x}_B, \ddot{y}_B, \ddot{z}_B$  (che rappresentano la variazione temporale delle velocità nel Body frame  $\dot{\mathbf{v}}_B$ ), si ottiene il set di equazioni relative al blocco della **Dinamica Traslazionale**:

$$\begin{cases} \ddot{x}_B = r\dot{y}_B - q\dot{z}_B - g \sin \theta \\ \ddot{y}_B = p\dot{z}_B - r\dot{x}_B + g \sin \phi \cos \theta \\ \ddot{z}_B = q\dot{x}_B - p\dot{y}_B + g \cos \phi \cos \theta - \frac{T}{m} \end{cases}$$

### 3.1.3 Cinematica Rotazionale

La cinematica rotazionale descrive la relazione tra i body rates del quadcopter:

$$\boldsymbol{\omega}_B = \begin{bmatrix} p \\ q \\ r \end{bmatrix}$$

e la variazione temporale degli angoli di Eulero:

$$\alpha = \begin{bmatrix} \phi \\ \theta \\ \psi \end{bmatrix}$$

La relazione cinematica è definita da:

$$\dot{\alpha} = E^{-1}(\phi, \theta) \omega_B$$

dove:

$$E^{-1}(\phi, \theta) = \begin{bmatrix} 1 & \sin \phi \tan \theta & \cos \phi \tan \theta \\ 0 & \cos \phi & -\sin \phi \\ 0 & \frac{\sin \phi}{\cos \theta} & \frac{\cos \phi}{\cos \theta} \end{bmatrix}$$

Esplicitando le tre equazioni:

$$\begin{cases} \dot{\phi} = p + \sin \phi \tan \theta q + \cos \phi \tan \theta r \\ \dot{\theta} = \cos \phi q - \sin \phi r \\ \dot{\psi} = \frac{\sin \phi}{\cos \theta} q + \frac{\cos \phi}{\cos \theta} r \end{cases}$$

*Remark* La parametrizzazione tramite angoli di Eulero utilizzata nel modello a 12 stati introduce una singolarità per  $\theta = \pm 90^\circ$ , nota come **Gimbal Lock**. Per evitare questo problema è possibile impiegare un modello a 13 stati basato sui *quaternioni*, che consente una rappresentazione dell'assetto priva di singolarità.

### 3.1.4 Dinamica Rotazionale

La dinamica rotazionale descrive l'evoluzione temporale delle velocità angolari del quadcopter nel body frame.

Definendo la matrice di inerzia:

$$J = \begin{bmatrix} J_{xx} & 0 & 0 \\ 0 & J_{yy} & 0 \\ 0 & 0 & J_{zz} \end{bmatrix}$$

le equazioni di Eulero per un corpo rigido assumono la forma:

$$J\dot{\omega}_B + \omega_B \times (J\omega_B) = \tau_B$$

dove:

$$\tau_B = \begin{bmatrix} L \\ M \\ N \end{bmatrix}$$

rappresenta il vettore delle coppie applicate al drone.

Esplicitando le tre equazioni dinamiche si ottiene:

$$\begin{cases} \dot{p} = \frac{1}{J_{xx}} (L - (J_{zz} - J_{yy}) qr) \\ \dot{q} = \frac{1}{J_{yy}} (M - (J_{xx} - J_{zz}) pr) \\ \dot{r} = \frac{1}{J_{zz}} (N - (J_{yy} - J_{xx}) pq) \end{cases}$$

## SUBSECTION 3.2

**Modello Dinamico a 13 Stati**

Il modello dinamico a 13 stati, implementato nella classe `A01_SimplifiedModel_quat.m`, assume la seguente dinamica:

$$\begin{cases} \dot{x}_I = (q_0^2 + q_1^2 - q_2^2 - q_3^2)\dot{x}_B + 2(q_1q_2 - q_0q_3)\dot{y}_B + 2(q_1q_3 + q_0q_2)\dot{z}_B \\ \dot{y}_I = 2(q_1q_2 + q_0q_3)\dot{x}_B + (q_0^2 - q_1^2 + q_2^2 - q_3^2)\dot{y}_B + 2(q_2q_3 - q_0q_1)\dot{z}_B \\ \dot{z}_I = 2(q_1q_3 - q_0q_2)\dot{x}_B + 2(q_2q_3 + q_0q_1)\dot{y}_B + (q_0^2 - q_1^2 - q_2^2 + q_3^2)\dot{z}_B \\ \ddot{x}_B = r\dot{y}_B - q_r\dot{z}_B + 2g(q_1q_3 - q_0q_2) \\ \ddot{y}_B = p\dot{z}_B - r\dot{x}_B + 2g(q_2q_3 + q_0q_1) \\ \ddot{z}_B = q_r\dot{x}_B - p\dot{y}_B + g(q_0^2 - q_1^2 - q_2^2 + q_3^2) - \frac{T}{m} \\ \dot{q}_0 = -\frac{1}{2}(q_1p + q_2q_r + q_3r) \\ \dot{q}_1 = \frac{1}{2}(q_0p + q_2r - q_3q_r) \\ \dot{q}_2 = \frac{1}{2}(q_0q_r + q_3p - q_1r) \\ \dot{q}_3 = \frac{1}{2}(q_0r + q_1q_r - q_2p) \\ \dot{p} = \frac{1}{J_{xx}}(L - (J_{zz} - J_{yy})q_r r) \\ \dot{q}_r = \frac{1}{J_{yy}}(M - (J_{xx} - J_{zz})pr) \\ \dot{r} = \frac{1}{J_{zz}}(N - (J_{yy} - J_{xx})pq_r) \end{cases}$$

**3.2.1 Cinematica della Posizione**

Il blocco relativo alla cinematica della posizione è stato ottenuto sfruttando la relazione tra la velocità espressa nel body frame e la velocità nel sistema inerziale:

$$\dot{\mathbf{p}}_I = \mathbf{R}(\mathbf{q}) \mathbf{v}_B$$

dove  $\mathbf{R}(\mathbf{q}) \in \mathbb{R}^{3 \times 3}$  è la matrice di rotazione associata al quaternione unitario  $\mathbf{q}$ , definito come:

$$\mathbf{q} = \begin{bmatrix} q_0 \\ q_1 \\ q_2 \\ q_3 \end{bmatrix} = \begin{bmatrix} q_0 \\ \mathbf{q}_v \end{bmatrix}, \quad \mathbf{q}_v = \begin{bmatrix} q_1 \\ q_2 \\ q_3 \end{bmatrix}$$

dove  $q_0$  rappresenta la parte scalare e  $\mathbf{q}_v$  la parte vettoriale del quaternione.

La matrice di rotazione associata al quaternione è data da:

$$\mathbf{R}(\mathbf{q}) = (q_0^2 - \|\mathbf{q}_v\|^2) \mathbf{I}_3 + 2\mathbf{q}_v \mathbf{q}_v^\top + 2q_0 \mathbf{S}(\mathbf{q}_v)$$

$$\mathbf{R}(\mathbf{q}) = \begin{bmatrix} q_0^2 + q_1^2 - q_2^2 - q_3^2 & 2(q_1q_2 - q_0q_3) & 2(q_1q_3 + q_0q_2) \\ 2(q_1q_2 + q_0q_3) & q_0^2 - q_1^2 + q_2^2 - q_3^2 & 2(q_2q_3 - q_0q_1) \\ 2(q_1q_3 - q_0q_2) & 2(q_2q_3 + q_0q_1) & q_0^2 - q_1^2 - q_2^2 + q_3^2 \end{bmatrix}$$

dove  $\mathbf{S}(\mathbf{q}_v)$  è la matrice antisimmetrica associata al prodotto vettoriale:

$$\mathbf{S}(\mathbf{q}_v) = \begin{bmatrix} 0 & -q_3 & q_2 \\ q_3 & 0 & -q_1 \\ -q_2 & q_1 & 0 \end{bmatrix}$$

### 3.2.2 Dinamica Traslazionale

Il blocco relativo alla dinamica traslazionale è stato ricavato a partire dalla seguente equazione del moto espressa nel body frame:

$$\mathbf{a}_B = -\boldsymbol{\omega}_B \times \mathbf{v}_B + \mathbf{g}_B + \frac{1}{m} \mathbf{F}_{T,B}$$

dove  $\mathbf{g}_B$  rappresenta il vettore gravitazionale espresso nel body frame.

Sviluppando l'espressione precedente si ottiene:

$$\begin{bmatrix} \ddot{x}_B \\ \ddot{y}_B \\ \ddot{z}_B \end{bmatrix} = \begin{bmatrix} q\dot{z}_B - r\dot{y}_B \\ r\dot{x}_B - p\dot{z}_B \\ p\dot{y}_B - q\dot{x}_B \end{bmatrix} + \begin{bmatrix} g_B^{(1)} \\ g_B^{(2)} \\ g_B^{(3)} \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ -\frac{T}{m} \end{bmatrix}$$

Il vettore gravitazionale nel sistema di riferimento del corpo è definito come:

$$\mathbf{g}_B = \begin{bmatrix} g_B^{(1)} \\ g_B^{(2)} \\ g_B^{(3)} \end{bmatrix} = \mathbf{R}(\mathbf{q})^\top \begin{bmatrix} 0 \\ 0 \\ g \end{bmatrix}$$

Le quantità  $g_B^{(1)}$ ,  $g_B^{(2)}$  e  $g_B^{(3)}$  rappresentano le componenti del vettore gravitazionale proiettato nel body frame.

### 3.2.3 Cinematica dell'Assetto tramite Quaternioni

L'evoluzione temporale dell'assetto è descritta mediante quaternioni unitari. Sfruttando il **prodotto di Hamilton** tra il quaternion di orientamento e la velocità angolare espressa nel body frame, si ottiene:

$$\frac{d}{dt} \begin{bmatrix} q_0 \\ q_1 \\ q_2 \\ q_3 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 0 & p & q & r \\ -p & 0 & -r & q \\ -q & r & 0 & -p \\ -r & -q & p & 0 \end{bmatrix} \begin{bmatrix} q_0 \\ q_1 \\ q_2 \\ q_3 \end{bmatrix}$$

Il blocco relativo alla dinamica rotazionale rimane coerente con quello adottato dal modello a 12 stati.

#### SUBSECTION 3.3

### Parametri

In questa prima implementazione il modello assume  $m = 0.270$  kg come massa complessiva del drone, mentre il braccio tra il centro del frame ed ogni motore è pari a  $b = 0.08$  m [4].

La configurazione adottata è di tipo **X**, con coppie di motori opposti aventi lo stesso verso di rotazione:

- motori 1 e 2: rotazione antioraria;
- motori 3 e 4: rotazione oraria.

Le dimensioni del frame sono:  $20 \times 20 \times 4$  cm.

La matrice di inerzia del corpo rigido viene assunta diagonale:

$$J = \text{diag}(J_{xx}, J_{yy}, J_{zz})$$

con valori:

$$J = \text{diag}(0.00307, 0.00307, 0.00239) \text{ kg m}^2$$

I parametri inerziali utilizzati nel modello sono stati ricavati da [2].

#### SUBSECTION 3.4

### Linearizzazione

---

Il modello dinamico non lineare viene linearizzato attorno ad un generico vettore operativo nella funzione `[A, B] = get_jacobians(x_lin, u_lin)`:

$$(x_{lin}, u_{lin})$$

Il sistema non lineare:

$$\dot{x} = f(x, u)$$

viene approssimato localmente tramite il modello linearizzato:

$$\delta \dot{x} = A \delta x + B \delta u$$

dove:

$$A = \left. \frac{\partial f(x, u)}{\partial x} \right|_{(x_{lin}, u_{lin})} \quad B = \left. \frac{\partial f(x, u)}{\partial u} \right|_{(x_{lin}, u_{lin})}$$

#### 3.4.1 Jacobiana rispetto agli Stati

La matrice  $A \in \mathbb{R}^{12 \times 12}$  è composta dalle derivate parziali:

$$A_{ij} = \frac{\partial \dot{x}_i}{\partial x_j}$$

In forma compatta:

$$A_{lin} = \begin{bmatrix} \frac{\partial \dot{p}_I}{\partial p_I} & \frac{\partial \dot{p}_I}{\partial v_B} & \frac{\partial \dot{p}_I}{\partial \alpha} & \frac{\partial \dot{p}_I}{\partial \omega_B} \\ \frac{\partial \dot{v}_B}{\partial p_I} & \frac{\partial \dot{v}_B}{\partial v_B} & \frac{\partial \dot{v}_B}{\partial \alpha} & \frac{\partial \dot{v}_B}{\partial \omega_B} \\ \frac{\partial \dot{\alpha}}{\partial p_I} & \frac{\partial \dot{\alpha}}{\partial v_B} & \frac{\partial \dot{\alpha}}{\partial \alpha} & \frac{\partial \dot{\alpha}}{\partial \omega_B} \\ \frac{\partial \dot{\omega}_B}{\partial p_I} & \frac{\partial \dot{\omega}_B}{\partial v_B} & \frac{\partial \dot{\omega}_B}{\partial \eta} & \frac{\partial \dot{\omega}_B}{\partial \omega_B} \end{bmatrix}$$

dove:

$$p_I = [x_I \ y_I \ z_I]^T, \quad v_B = [\dot{x}_B \ \dot{y}_B \ \dot{z}_B]^T, \quad \alpha = [\phi \ \theta \ \psi]^T, \quad \omega_B = [p \ q \ r]^T$$

#### 3.4.2 Jacobiana rispetto agli Input

La matrice jacobiana degli input  $B \in \mathbb{R}^{12 \times 4}$  è definita come:

$$B_{ij} = \frac{\partial \dot{x}_i}{\partial u_j}$$

Nel modello considerato, la matrice linearizzata rispetto agli input assume la forma:

$$B_{lin} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ -\frac{1}{m} & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & \frac{1}{J_{xx}} & 0 & 0 \\ 0 & 0 & \frac{1}{J_{yy}} & 0 \\ 0 & 0 & 0 & \frac{1}{J_{zz}} \end{bmatrix}$$

Le componenti non nulle della Jacobiana sono quindi:

$$\frac{\partial \ddot{z}_B}{\partial T} = -\frac{1}{m}, \quad \frac{\partial \dot{p}}{\partial L} = \frac{1}{J_{xx}}, \quad \frac{\partial \dot{q}}{\partial M} = \frac{1}{J_{yy}}, \quad \frac{\partial \dot{r}}{\partial N} = \frac{1}{J_{zz}}$$

Le analoghe considerazioni sulla linearizzazione valgono anche per il modello descritto dai quaternioni a 13 stati.

#### SECTION 4

## Flatness Map

---

Il quadcopter considerato appartiene alla classe dei sistemi **differenzialmente piatti** [5]. Una possibile scelta delle uscite piatte è costituita da:

$$\sigma = \begin{bmatrix} x_I \\ y_I \\ z_I \\ \psi \end{bmatrix}$$

dove le prime tre componenti rappresentano la posizione del drone nel sistema di riferimento inerziale, mentre  $\psi$  identifica l'angolo di yaw.

*Remark*

La proprietà di **differential flatness** garantisce che l'intero stato del sistema e gli input possano essere espressi in forma algebrica come funzione delle uscite piatte e di un numero finito delle loro derivate temporali, senza la necessità di integrare il modello dinamico [6].

In particolare, esistono delle mappe tali che:

$$x = h(\sigma, \dot{\sigma}, \ddot{\sigma}, \sigma^{(3)}) \quad u = i(\sigma, \dot{\sigma}, \ddot{\sigma}, \sigma^{(3)}, \sigma^{(4)})$$

dove:

- $\sigma^{(3)}$  rappresenta il **jerk**, ovvero la terza derivata temporale della traiettoria;
- $\sigma^{(4)}$  rappresenta lo **snap**, ovvero la quarta derivata temporale della traiettoria.

La formulazione adottata in questo progetto segue la teoria della differential flatness applicata ai quadrotori riportata in [5, 7].

## SUBSECTION 4.1

**State Reconstruction**

Per quanto riguarda le posizioni nel sistema di riferimento inerziale, le prime tre uscite piatte coincidono direttamente con gli stati di posizione del modello:

$$p_I = \begin{bmatrix} x_I \\ y_I \\ z_I \end{bmatrix} = \sigma_{1:3}$$

Analogamente, la velocità nel frame inerziale è ottenuta dalla prima derivata delle uscite piatte:

$$v_I = \begin{bmatrix} \dot{x}_I \\ \dot{y}_I \\ \dot{z}_I \end{bmatrix} = \dot{\sigma}_{1:3}$$

Per ricostruire le velocità espresse nel body frame è necessario determinare l'assetto del drone, ovvero la matrice di rotazione  $R$  che descrive l'orientamento del body frame rispetto al sistema di riferimento inerziale.

A partire dall'uscita piatta associata allo yaw viene definito il vettore di heading orizzontale:

$$x_C = \begin{bmatrix} \cos \psi \\ \sin \psi \\ 0 \end{bmatrix}$$

La direzione dell'asse verticale del drone viene invece ricavata dalla forza netta richiesta dalla traiettoria. Applicando la seconda legge di Newton nel sistema inerziale:

$$m a_I = m g e_z + R \begin{bmatrix} 0 \\ 0 \\ -T \end{bmatrix}$$

si definisce il vettore:

$$F_{net} = m(a_I - g e_z), \quad e_z = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

La norma di tale vettore corrisponde al thrust richiesto e la direzione dell'asse  $z_B$  nel frame inerziale è:

$$T = \|F_{net}\|, \quad z_B = -\frac{F_{net}}{\|F_{net}\|}$$

Una volta noti  $x_C$  e  $z_B$ , gli altri assi del body frame vengono ricostruiti:

$$y_B = \frac{z_B \times x_C}{\|z_B \times x_C\|}, \quad x_B = y_B \times z_B, \quad R = [x_B \ y_B \ z_B]$$

Dalla matrice  $R$  è possibile ricavare gli angoli di Eulero secondo la convenzione Z-Y-X:

$$\phi = \text{atan2}(R_{32}, R_{33}), \quad \theta = \arcsin(-R_{31}), \quad \psi = \text{atan2}(R_{21}, R_{11})$$

Per la versione a 13 stati del modello, viene usato il **metodo di Shepperd** per garantire stabilità numerica e ricostruire l'assetto in forma di quaternioni  $q = [q_0, q_1, q_2, q_3]$  direttamente dalla matrice di rotazione  $R$ .

Le velocità lineari nel body frame sono ottenute mediante la trasformazione:

$$v_B = R^T v_I$$

Per la ricostruzione delle velocità angolari è necessario derivare temporalmente la matrice di rotazione  $R$ , operazione che richiede la conoscenza del **jerk** della traiettoria. Dalla dinamica traslazionale si definisce:

$$F_{net} = m(a_I - g e_z), \quad \dot{F}_{net} = m j_I, \quad j_I = \begin{bmatrix} \ddot{x}_I \\ \ddot{y}_I \\ \ddot{z}_I \end{bmatrix}$$

Poiché  $z_B = -F_{net}/\|F_{net}\|$ , è possibile derivarne la variazione temporale ottenendo  $\dot{z}_B$ . Derivando il vettore di heading orizzontale si ottiene:

$$\dot{x}_C = \dot{\psi} \begin{bmatrix} -\sin \psi \\ \cos \psi \\ 0 \end{bmatrix}$$

Derivando le relazioni che definiscono  $y_B$  e  $x_B$  si ottengono  $\dot{y}_B$  e  $\dot{x}_B$ . La derivata della matrice di rotazione risulta quindi:

$$\dot{R} = [\dot{x}_B \ \dot{y}_B \ \dot{z}_B]$$

La velocità angolare del corpo viene ricostruita mediante la relazione cinematica:

$$\hat{\omega} = R^T \dot{R}$$

dove  $\hat{\omega}$  è la matrice antisimmetrica associata al vettore delle velocità angolari. Le componenti dei body rates vengono estratte come:

$$p = \hat{\omega}_{32}, \quad q = \hat{\omega}_{13}, \quad r = \hat{\omega}_{21}$$

In questo modo il vettore delle velocità angolari:

$$\omega_B = \begin{bmatrix} p \\ q \\ r \end{bmatrix}$$

viene ottenuto direttamente dalle uscite piatte e dalle loro derivate fino al jerk, senza integrare le equazioni rotazionali del modello.

Lo stato completo ricostruito assume quindi la forma:

$$x = \begin{bmatrix} p_I \\ v_B \\ \phi \\ \theta \\ \psi \\ p \\ q \\ r \end{bmatrix} \in \mathbb{R}^{12}, \quad x = \begin{bmatrix} p_I \\ v_B \\ q_0 \\ q_1 \\ q_2 \\ q_3 \\ p \\ q \\ r \end{bmatrix} \in \mathbb{R}^{13}$$

#### SUBSECTION 4.2

### Input Reconstruction

La ricostruzione delle coppie richiede la conoscenza delle accelerazioni angolari del corpo. Per questo motivo è necessario derivare ulteriormente la cinematica rotazionale fino ad ottenere  $\dot{\omega}_B$ , operazione che richiede la disponibilità dello **snap** della traiettoria.



A partire dallo snap viene calcolata la derivata seconda della matrice di rotazione  $\ddot{R}$ . Ricordando la relazione cinematica  $\dot{\omega} = R^T \ddot{R}$ , la sua derivata temporale risulta:

$$\dot{\omega} = R^T \ddot{R} - \dot{\omega}^2$$

Dalla matrice antisimmetrica  $\dot{\omega}$  si ricavano le accelerazioni angolari:

$$\dot{\omega}_B = \begin{bmatrix} \dot{p} \\ \dot{q} \\ \dot{r} \end{bmatrix}$$

Una volta noti  $\omega_B = [p, q, r]^T$  e  $\dot{\omega}_B = [\dot{p}, \dot{q}, \dot{r}]^T$ , le coppie richieste vengono ottenute dalle equazioni di Newton-Eulero:

$$J\dot{\omega}_B + \omega_B \times (J\omega_B) = \tau_B, \quad \tau_B = \begin{bmatrix} L \\ M \\ N \end{bmatrix}$$

Pertanto:

$$\tau_B = J\dot{\omega}_B + \omega_B \times (J\omega_B)$$

Le componenti del vettore delle coppie forniscono direttamente gli input:

$$L = \tau_{B,1}, \quad M = \tau_{B,2}, \quad N = \tau_{B,3}$$

Il thrust collettivo è invece ottenuto dalla norma della forza netta richiesta dalla traiettoria:

$$T = \|F_{net}\|$$

L'input ricostruito, in entrambe le rappresentazioni, assume la forma:

$$u = \begin{bmatrix} T \\ L \\ M \\ N \end{bmatrix}$$

# Definizione delle Traiettorie

PART

III

SECTION 5

## Test sui Gradi di Libertà (DoF)

Una volta definite le uscite piatte e la relativa mappa di ricostruzione, è possibile generare **traiettorie analiticamente feasible** direttamente nello spazio delle uscite piatte. Prima di procedere con la validazione di traiettorie complesse e con l'implementazione del controllore MPC, sono stati eseguiti una serie di test elementari finalizzati alla verifica dei singoli gradi di libertà del modello.

*Remark*

**Convenzioni di Segno e Orientamento (NED / FRD).** Le simulazioni e le visualizzazioni grafiche adottano i sistemi di riferimento **NED** (*North-East-Down*, inerziale) e **FRD** (*Forward-Right-Down*, body frame):

- **Beccheggio – Pitch ( $\theta$ ):**

–  $\theta > 0 \rightarrow$  muso verso l'alto (rotori 1–3 alzati)  $\rightarrow$  **decelerazione / frenata**;

- $\theta < 0 \rightarrow$  muso verso il basso  $\rightarrow$  **avanzamento**.
- **Rollio – Roll** ( $\phi$ ):
  - $\phi > 0 \rightarrow$  inclinazione a destra (rotori 1–4 giù)  $\rightarrow$  **traslazione a destra**;
  - $\phi < 0 \rightarrow$  inclinazione a sinistra (rotori 2–3 giù)  $\rightarrow$  **traslazione a sinistra**.
- **Imbardata – Yaw** ( $\psi$ ):
  - $\psi > 0 \rightarrow$  rotazione in **senso orario** (vista dall'alto);
  - $\psi < 0 \rightarrow$  rotazione in **senso antiorario**.
- **Dinamica Verticale** ( $z_{\text{NED}}$ ): poiché l'asse  $Z$  è rivolto verso il basso, un moto di **salita** corrisponde a un valore di quota e a una forza di spinta (thrust) di segno **negativo**.

Le traiettorie di test sono implementate nelle classi `ShapeRollTest`, `ShapePitchTest`, `ShapeThrustTest` e `ShapeYawTest`, e sono progettate per eccitare una sola componente delle uscite piatte, mantenendo costanti tutte le altre.

Gli obiettivi principali di tali simulazioni sono:

- ispezionare visivamente il comportamento del drone all'interno del simulatore;
- verificare la corretta associazione tra traiettoria imposta e movimento generato;
- validare i segni adottati nei sistemi di riferimento NED e FRD;
- verificare la coerenza degli angoli di assetto ricostruiti tramite la `FlatnessMap`;
- controllare la correttezza dei grafici e delle convenzioni adottate;
- verificare che il modello dinamico produca le reazioni attese in corrispondenza delle diverse componenti del moto.

Tutte le traiettorie vengono inizializzate in condizioni tali che:

$$x(0) = \begin{bmatrix} 0 \\ 0 \\ z_0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \in \mathbb{R}^{12}$$

ovvero posizione iniziale assegnata, velocità lineari nulle, angoli di Eulero nulli e body rates nulli.

## SUBSECTION 5.1

**Test Longitudinale (Pitch)**

Per il test longitudinale viene imposta una variazione della sola coordinata  $x_I$  (varia l'angolo di pitch  $\theta$ , definito come rotazione attorno all'asse  $y$ ), mantenendo costanti quota e yaw:

$$\sigma(t) = \begin{bmatrix} x(t) \\ 0 \\ z_0 \\ 0 \end{bmatrix}, \quad x(t) = \frac{A}{8} (1 - \cos(\omega t))^3$$

La traiettoria è una funzione periodica di classe  $C^4$ , parametrizzata da un'ampiezza  $A$ , una quota costante  $z_0$  e una velocità angolare  $\omega$ . L'accelerazione richiesta lungo l'asse  $x_I$  induce principalmente variazioni dell'angolo di Pitch, consentendo di verificare la corretta risposta del modello alle manovre longitudinali.

## SUBSECTION 5.2

**Test Laterale (Roll)**

Per il test laterale viene imposta una variazione della sola coordinata  $y_I$  (varia l'angolo di roll  $\phi$ , definito come rotazione attorno all'asse  $x$ ):

$$\sigma(t) = \begin{bmatrix} 0 \\ y(t) \\ z_0 \\ 0 \end{bmatrix}, \quad y(t) = \frac{A}{8} (1 - \cos(\omega t))^3$$

La traiettoria è di classe  $C^4$  e risulta parametrizzata da  $A$ ,  $z_0$  e  $\omega$ . Il movimento generato lungo l'asse laterale richiede prevalentemente variazioni dell'angolo di Roll e permette di verificare la corretta ricostruzione dell'assetto durante manovre trasversali.

## SUBSECTION 5.3

**Test Verticale (Thrust)**

Per il test verticale viene modificata esclusivamente la quota, mantenendo costante l'orientamento del veicolo:

$$\sigma(t) = \begin{bmatrix} 0 \\ 0 \\ z(t) \\ 0 \end{bmatrix}, \quad z(t) = z_0 - \frac{H}{2} (1 - \cos(\omega t))$$

La traiettoria è parametrizzata dalla quota iniziale  $z_0$ , dall'escursione verticale  $H$  e dalla velocità angolare  $\omega$ . Questo test consente di validare la dinamica verticale del modello e la corretta generazione del thrust collettivo necessario a compensare la gravità e a produrre le accelerazioni richieste.

## SUBSECTION 5.4

**Test Direzionale (Yaw)**

Per il test direzionale viene mantenuta costante la posizione e viene eccitata esclusivamente la componente di yaw:

$$\sigma(t) = \begin{bmatrix} 0 \\ 0 \\ z_0 \\ \psi(t) \end{bmatrix}, \quad \psi(t) = \frac{A}{8} (1 - \cos(\omega t))^3$$

La traiettoria è parametrizzata dalla quota costante  $z_0$ , dall'ampiezza angolare  $A$  e dalla velocità angolare  $\omega$ . Poiché la posizione rimane invariata, il test isola la sola dinamica di imbardata, consentendo di verificare la corretta ricostruzione dell'orientamento, dei body rates e della coppia di yaw richiesta dalla Flatness Map.

## SECTION 6

**Traiettorie Complesse**

Le traiettorie complesse definite comprendono il **cerchio**, la **lemniscata (di Geroni)** e la **spirale**. Queste tre tipologie rappresentano movimenti nello spazio con crescente livello di complessità dinamica e geometrica, passando da un moto periodico semplice e uniforme (cerchio), a una traiettoria con intersezione e variazioni di curvatura (lemniscata), fino a un moto elicoidale con variazione della quota nel tempo (spirale).

## SUBSECTION 6.1

**Traiettoria Circolare**

La traiettoria circolare descrive un moto uniforme nel piano  $XY$  con quota costante, con flat outputs  $\sigma = [x_I, y_I, z_I, \psi]^T$ .

**Posizione**

$$\begin{cases} x(t) = R \cos(\omega t) \\ y(t) = R \sin(\omega t) \\ z(t) = h \\ \psi(t) = \omega t + \frac{\pi}{2} \end{cases}$$

**Velocità e Accelerazione**

$$\begin{cases} \dot{x}(t) = -R\omega \sin(\omega t), & \ddot{x}(t) = -R\omega^2 \cos(\omega t) \\ \dot{y}(t) = R\omega \cos(\omega t), & \ddot{y}(t) = -R\omega^2 \sin(\omega t) \\ \dot{z}(t) = 0, & \ddot{z}(t) = 0 \\ \dot{\psi}(t) = \omega, & \ddot{\psi}(t) = 0 \end{cases}$$

**Jerk e Snap**

$$\begin{cases} \ddot{\dot{x}}(t) = R\omega^3 \sin(\omega t), & x^{(4)}(t) = R\omega^4 \cos(\omega t) \\ \ddot{\dot{y}}(t) = -R\omega^3 \cos(\omega t), & y^{(4)}(t) = R\omega^4 \sin(\omega t) \\ \ddot{\dot{z}}(t) = 0, & z^{(4)}(t) = 0 \\ \ddot{\dot{\psi}}(t) = 0, & \psi^{(4)}(t) = 0 \end{cases}$$

con  $R = 1.0$  m,  $\omega = 1.0$  rad/s,  $h = -1.0$  m (NED). L'angolo di yaw è allineato con la direzione della velocità tangenziale. La durata della traiettoria è pari a un giro completo:

$$T_{\text{circle}} = \frac{2\pi}{\omega}$$

## SUBSECTION 6.2

**Lemniscata di Geroni**

La traiettoria a lemniscata di Geroni descrive un moto piano nel quale la proiezione nel piano  $XY$  genera una figura a “8”, con flat outputs  $\sigma = [x_I, y_I, z_I, \psi]^T$ .

**Posizione**

$$\begin{cases} x(t) = R \cos(\omega t) + c_x \\ y(t) = \frac{R}{2} \sin(2\omega t) + c_y \\ z(t) = h \\ \psi(t) = \text{atan2}(\dot{y}(t), \dot{x}(t)) \end{cases}$$

**Velocità e Accelerazione**

$$\begin{cases} \dot{x}(t) = -R\omega \sin(\omega t), & \ddot{x}(t) = -R\omega^2 \cos(\omega t) \\ \dot{y}(t) = R\omega \cos(2\omega t), & \ddot{y}(t) = -2R\omega^2 \sin(2\omega t) \\ \dot{z}(t) = 0, & \ddot{z}(t) = 0 \\ \dot{\psi}(t) = \frac{\dot{x}\ddot{y} - \dot{y}\ddot{x}}{\dot{x}^2 + \dot{y}^2}, & \ddot{\psi}(t) = \frac{d}{dt} \left( \frac{\dot{x}\ddot{y} - \dot{y}\ddot{x}}{\dot{x}^2 + \dot{y}^2} \right) \end{cases}$$

**Jerk e Snap**

$$\begin{cases} \ddot{\dot{x}}(t) = R\omega^3 \sin(\omega t), & x^{(4)}(t) = R\omega^4 \cos(\omega t) \\ \ddot{\dot{y}}(t) = -4R\omega^3 \cos(2\omega t), & y^{(4)}(t) = 8R\omega^4 \sin(2\omega t) \\ \ddot{\dot{z}}(t) = 0, & z^{(4)}(t) = 0 \\ \ddot{\dot{\psi}}(t) = 0, & \psi^{(4)}(t) = 0 \end{cases}$$

con  $R = 1.0$  m,  $\omega = 0.5$  rad/s,  $h = -1.0$  m (NED),  $(c_x, c_y) = (0, 0)$ . La durata della traiettoria è:

$$T_{\text{lemniscate}} = \frac{2\pi}{\omega}$$

*Remark*

La Lemniscata di Geroni (**Lemniscate2**) è stata utilizzata nelle simulazioni in closed-loop in quanto riduce la divergenza, eliminando i picchi di Jerk/Snap indotti dal denominatore di Bernoulli. Ridurre  $\omega$  a 0.5 rad/s abbassa l'ampiezza dello Snap di 16 volte ( $\omega^4$ ), stabilizzando l'integrazione PWC. Per la ricostruzione dell'input è richiesto di derivare lo yaw fino al secondo ordine ( $\ddot{\psi}(t)$ ).

## SUBSECTION 6.3

**Traiettoria Spirale**

La traiettoria a spirale descrive un moto circolare nel piano  $XY$  con raggio ellittico e quota variabile linearmente nel tempo, con flat outputs  $\sigma = [x_I, y_I, z_I, \psi]^T$ .

**Posizione**

$$\begin{cases} x(t) = R_x \cos(\omega t) \\ y(t) = R_y \sin(\omega t) \\ z(t) = z_0 - v_z t \\ \psi(t) = \text{atan2}(\dot{y}(t), \dot{x}(t)) \end{cases}$$

**Velocità e Accelerazione**

$$\begin{cases} \dot{x}(t) = -R_x \omega \sin(\omega t), & \ddot{x}(t) = -R_x \omega^2 \cos(\omega t) \\ \dot{y}(t) = R_y \omega \cos(\omega t), & \ddot{y}(t) = -R_y \omega^2 \sin(\omega t) \\ \dot{z}(t) = -v_z, & \ddot{z}(t) = 0 \\ \dot{\psi}(t) = \frac{\dot{x}\ddot{y} - \dot{y}\ddot{x}}{\dot{x}^2 + \dot{y}^2}, & \ddot{\psi}(t) = \frac{d}{dt} \left( \frac{\dot{x}\ddot{y} - \dot{y}\ddot{x}}{\dot{x}^2 + \dot{y}^2} \right) \end{cases}$$

**Jerk e Snap**

$$\begin{cases} \ddot{\dot{x}}(t) = R_x \omega^3 \sin(\omega t), & x^{(4)}(t) = R_x \omega^4 \cos(\omega t) \\ \ddot{\dot{y}}(t) = -R_y \omega^3 \cos(\omega t), & y^{(4)}(t) = R_y \omega^4 \sin(\omega t) \\ \ddot{\dot{z}}(t) = 0, & z^{(4)}(t) = 0 \\ \ddot{\dot{\psi}}(t) = 0, & \psi^{(4)}(t) = 0 \end{cases}$$

con  $R_x = 2.0$  m,  $R_y = 1.0$  m,  $\omega = 0.5$  rad/s,  $z_0 = -0.5$  m (NED),  $v_z = 0.2$  m/s.

Poiché  $R_x \neq R_y$ , la traiettoria è ellittica e la direzione tangente non è esprimibile in forma chiusa semplice: il quarto flat output è quindi definito come  $\psi(t) = \text{atan2}(\dot{y}(t), \dot{x}(t))$ . La quota decresce linearmente nel tempo (moto elicoidale discendente); in NED il drone aumenta di quota. La durata della traiettoria è:

$$T_{\text{spiral}} = \frac{4\pi}{\omega}$$

## SECTION 7

**Open-Loop Test**

Prima di eseguire la simulazione di controllo MPC, le traiettorie sono state testate in open-loop, **integrando gli input e gli stati campionati tramite la flatness map** mediante l'integratore numerico `ode45`.

L'obiettivo di questi test preliminari è verificare la correttezza della derivazione del modello e della generazione delle traiettorie, assicurando che eventuali errori siano contenuti. In particolare, si controlla la coerenza tra le grandezze ricostruite (stati e derivate) e la dinamica attesa, al fine di validare l'intero processo di generazione dei reference prima dell'introduzione del controllore MPC in closed-loop.

*Remark*

**Rappresentazione d'assetto e normalizzazione dei quaternioni.** Nel modello a 13 stati, la dinamica non-lineare integra una procedura di normalizzazione periodica del quaternion per prevenire il drift numerico e preservare il vincolo di norma unitaria, in conformità con la letteratura sul controllo d'assetto basato su quaternioni. Nei test in open-loop, è possibile selezionare la rappresentazione dell'orientamento desiderata (quaternioni o angoli di Eulero) configurando il flag `use_quaternions`.

## SUBSECTION 7.1

## Open-Loop Test — Cerchio

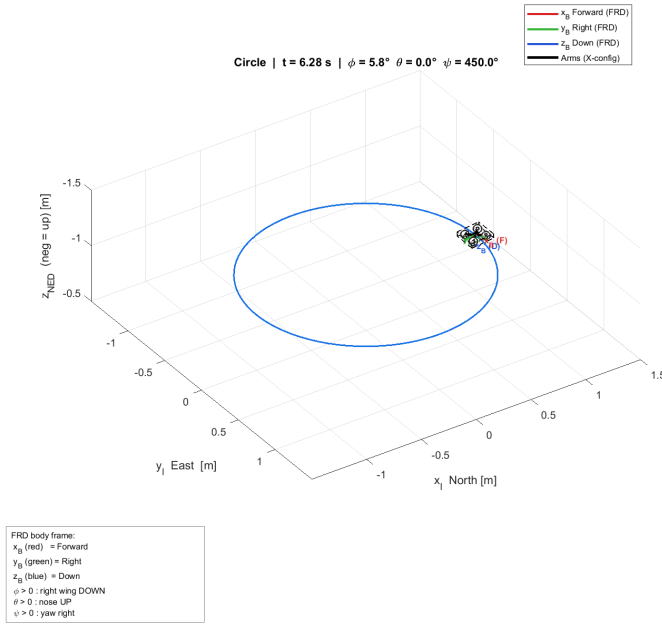


Figure 2. Traiettoria circolare: visualizzazione 3D del volo in open-loop.

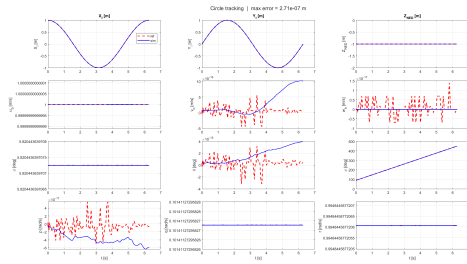


Figure 3. Cerchio: evoluzione temporale degli stati.

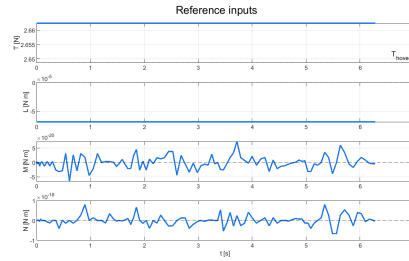


Figure 4. Cerchio: input di controllo di riferimento.

## SUBSECTION 7.2

## Open-Loop Test — Lemniscata

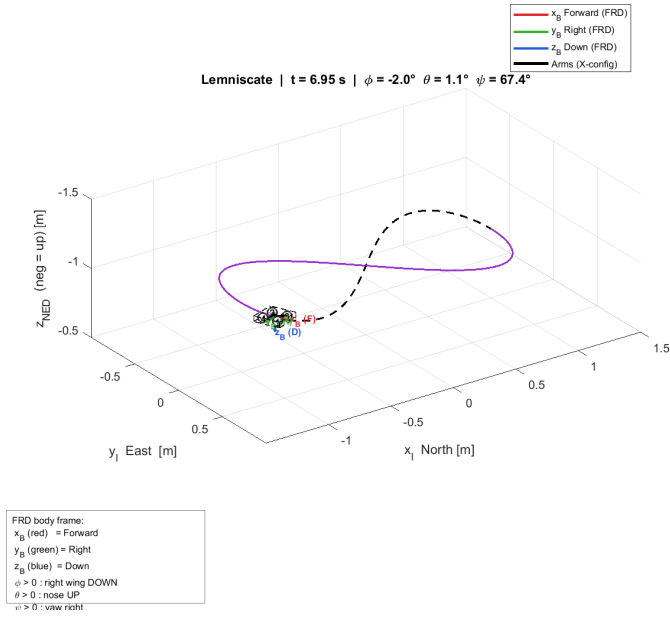


Figure 5. Lemniscata: visualizzazione 3D del volo in open-loop.

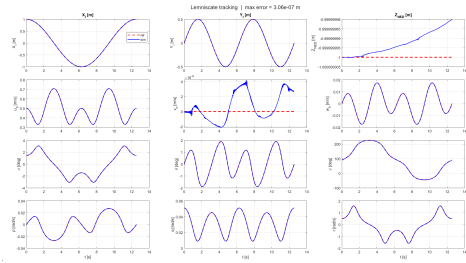


Figure 6. Lemniscata: evoluzione temporale degli stati.

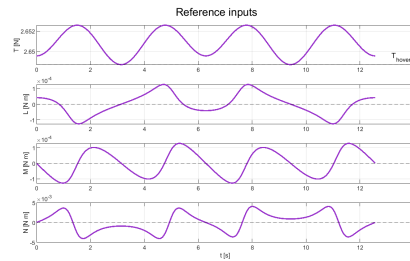


Figure 7. Lemniscata: input di controllo di riferimento.



## SUBSECTION 7.3

## Open-Loop Test — Spirale

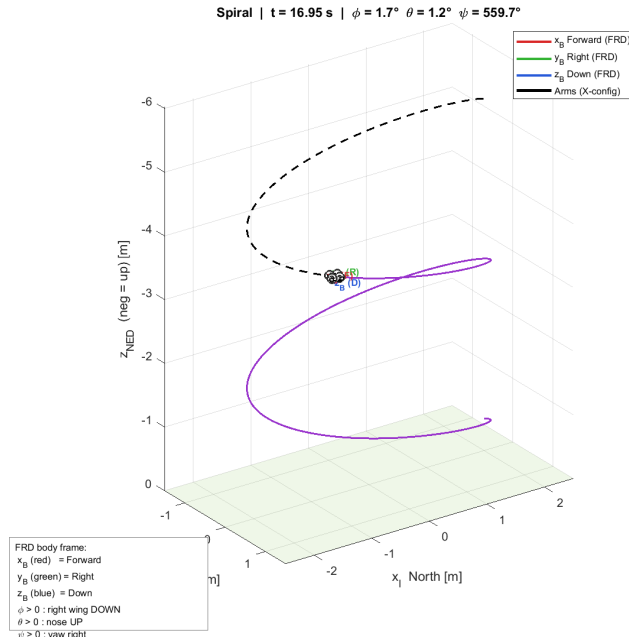


Figure 8. Spirale: visualizzazione 3D del volo in open-loop.

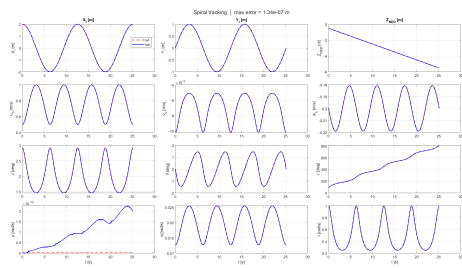


Figure 9. Spirale: evoluzione temporale degli stati.

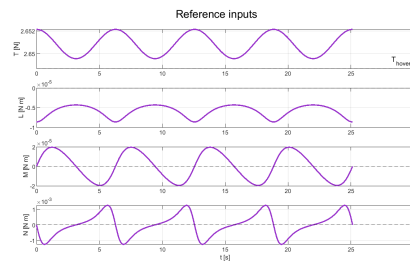


Figure 10. Spirale: input di controllo di riferimento.

## SUBSECTION 7.4

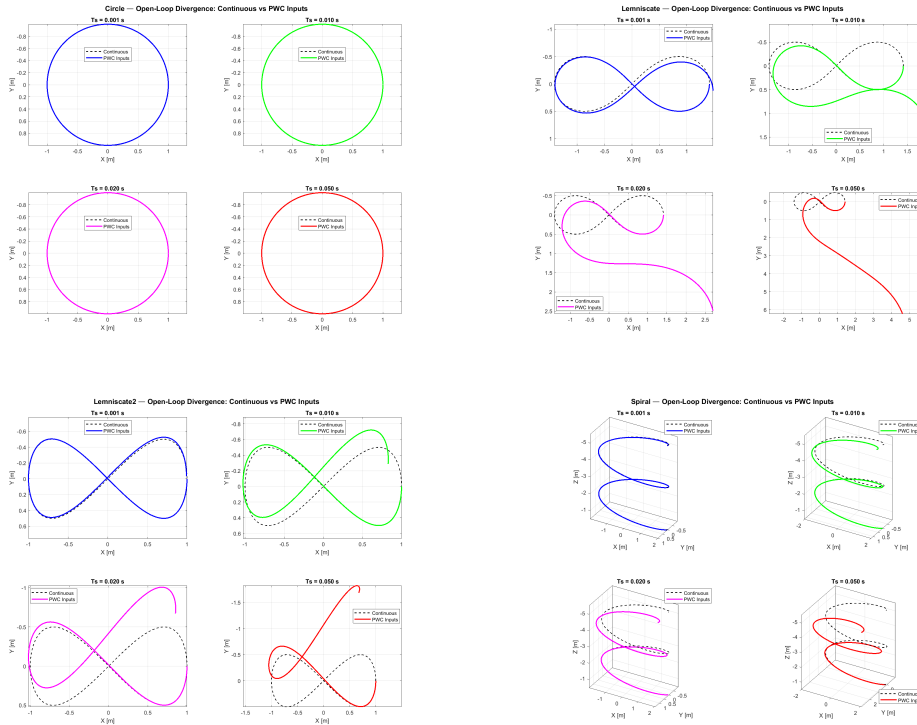
**Open-Loop: Riferimento Continuo vs Riferimento con Input PWC**

Questo test analizza l'effetto della discretizzazione temporale degli input sulla risposta in open-loop del sistema non-lineare, mettendo a confronto:

- **Riferimento Continuo (linea tratteggiata nera):** la traiettoria di riferimento ideale, i cui stati e input sono calcolati analiticamente istante per istante tramite la Flatness Map.
- **Modelli a input PWC (linee colorate):** la traiettoria ottenuta integrando le equazioni dinamiche non-lineari del drone applicando in modo costante a tratti (Piece-Wise Constant) gli input campionati dalla Flatness Map ad ogni sampling time  $T_s$ .

La traiettoria generata tramite l'integrazione con input PWC costituisce la traiettoria di riferimento effettiva che l'algoritmo MPC dovrà inseguire in closed-loop. Lo studio valuta la robustezza della traiettoria al variare del tempo di campionamento:

$$T_s \in \{0.001, 0.010, 0.020, 0.050\} \text{ s}$$



**Figure 11.** Confronto tra la traiettoria ideale continua (tratteggiata) e la traiettoria con input PWC (colorata) per diversi intervalli di campionamento  $T_s \in \{0.001, 0.010, 0.020, 0.050\}$  s. In alto a sinistra: **Cerchio**. In alto a destra: **Lemniscata di Bernoulli**. In basso a sinistra: **Lemniscata di Geronon**. In basso a destra: **Spirale**.

# Implementazione MPC

SECTION 8

## LTV-MPC

Per il tracking delle traiettorie di riferimento è stato implementato un controllore (**LTV-MPC**) basato sulla linearizzazione locale del modello non lineare del quadcopter lungo una traiettoria nominale. Ad ogni istante di campionamento viene risolto un problema di ottimizzazione quadratica (QP) su un orizzonte finito, applicando solamente il primo input ottimo secondo la strategia **Receding Horizon**.

SUBSECTION 8.1

### Parametri di Controllo

Il controllore utilizza un tempo di campionamento pari a  $T_s = 0.05$  s. In una implementazione reale, questo parametro dovrebbe tenere conto del tempo di calcolo del QP, della latenza degli ESC e della frequenza di acquisizione dei dati sullo stato.

In simulazione, il parametro  $T_s$  è modificabile per modellare ritardi e latenze fisiche del sistema reale, quali il tempo di calcolo del solutore, i tempi di risposta degli ESC o la frequenza di campionamento dell'IMU.

L'orizzonte di predizione è impostato a  $p = 18$ .

La funzione costo quadratica utilizza le matrici diagonali:

$$Q = Q_f = \text{diag}(50, 50, 50, 8, 8, 12, 8, 8, 10, 1, 1, 1)$$

e

$$R = \text{diag}(2, 2, 2, 2)$$

Questi valori sono stati inizializzati seguendo la struttura riportata in [3], assieme alla formulazione LTV-MPC adottata nel progetto.

*Remark*

I valori di  $Q$ ,  $Q_f$  e  $R$  richiedono una fase di tuning sperimentale dipendente dal modello utilizzato e dalla traiettoria di riferimento. In questa implementazione è stata assegnata una maggiore penalizzazione agli errori di posizione nel sistema di riferimento inerziale  $p_I$  e all'angolo di yaw  $\psi$ . Una minore enfasi è stata invece attribuita alle velocità nel body frame e alle velocità angolari. La matrice  $R$  è stata inizializzata in modo uniforme, assegnando lo stesso costo a tutti gli input.

I vincoli sugli stati e sugli ingressi sono introdotti direttamente all'interno del problema di ottimizzazione tramite limiti inferiori e superiori (LB e UB). In particolare, per il modello a 12 stati sono stati imposti i seguenti vincoli:

$$-3 \leq \dot{x}_B, \dot{y}_B \leq 3 \text{ m/s}$$

$$-3 \leq \dot{z}_B \leq 1 \text{ m/s}$$

$$-35^\circ \leq \phi, \theta \leq 35^\circ$$

$$-600^\circ/\text{s} \leq p, q, r \leq 600^\circ/\text{s}$$

mentre le posizioni  $(x_I, y_I, z_I)$  e l'angolo di yaw  $\psi$  non sono stati vincolati. Questi valori sono stati ricavati all'interno della tabella **5.3** nella tesi [4].

*Remark* **Formulazione dei vincoli nel modello a 13 stati:** in questa prima analisi, i vincoli sui quaternioni  $q$  sono stati **omessi** per validare la stabilità del loop di controllo ed evitare fenomeni di infattibilità numerica del solutore QP.

Per quanto riguarda gli input di controllo, sono state considerate le seguenti saturazioni come stime iniziali (da modificare e ricavare):

$$\begin{aligned} 0 &\leq T \leq 2\,mg \approx 5.30\text{ N} \\ -0.15 &\leq L, M \leq 0.15\text{ N m} \\ -0.05 &\leq N \leq 0.05\text{ N m} \end{aligned}$$

## SUBSECTION 8.2

### Strategia di Ottimizzazione

---

Ad ogni iterazione il metodo **solve** esegue una procedura di **linearizzazione**. Per ciascun passo dell'orizzonte vengono calcolate le matrici Jacobiane del modello:

$$A_{lin} = \frac{\partial f}{\partial x}, \quad B_{lin} = \frac{\partial f}{\partial u}$$

linearizzate attorno alla traiettoria nominale corrente, stato misurato di  $x \in \mathbb{R}^{12}$  o  $\mathbb{R}^{13}$ .

Successivamente il modello viene discretizzato mediante integrazione di Eulero, calcolando:

$$x_{k+1} = A_d x_k + B_d u_k + d_k$$

dove il termine affine

$$d_k = x_{k+1}^{NL} - (A_d x_k + B_d u_k)$$

compensa l'errore introdotto dalla linearizzazione locale. Le matrici discretizzate vengono quindi utilizzate per costruire il problema QP da risolvere ad ogni istante.

## SUBSECTION 8.3

### Costruzione del Problema QP

---

La generazione delle matrici di ottimizzazione è affidata alla classe **QPBuilder**.

Il vettore delle variabili decisionali è definito come:

$$Z = [x_0 \ u_0 \ x_1 \ u_1 \ \cdots \ u_{N-1} \ x_N]^T$$

e il problema viene formulato nella forma sparse:

$$\min_Z \frac{1}{2} Z^T H Z + f^T Z$$

soggetto a:

$$A_{eq} Z = b_{eq}$$

e ai vincoli:

$$lb \leq Z \leq ub$$

La matrice Hessiana  $H$  è costruita come matrice blocco diagonale a partire da  $Q$ ,  $R$  e  $Q_f$ , mentre il termine lineare  $f$  incorpora le traiettorie di riferimento degli stati e degli input. I vincoli dinamici sono imposti tramite le equazioni di uguaglianza:

$$x_{k+1} = A_d x_k + B_d u_k + d_k$$

per ogni passo dell'orizzonte. L'ottimizzazione viene infine risolta tramite il solver `quadprog`.

#### SUBSECTION 8.4

### Warm Start

---

Una volta ottenuta la soluzione ottima, vengono estratti gli stati e gli input predetti. Solamente il primo ingresso

$$u_0^* \in \mathbb{R}^4$$

viene applicato al sistema non-lineare.

La soluzione ottima viene quindi traslata di un passo e riutilizzata come nuova traiettoria nominale per l'iterazione successiva, implementando una strategia di **warm start** che riduce il numero di iterazioni richieste dal solver e migliora la continuità della soluzione lungo la simulazione.

#### SECTION 9

### Simulazione in Closed-Loop

---

La validazione del controllore LTV-MPC è stata effettuata in `MPCSimulationLoop.m`.

#### SUBSECTION 9.1

### Traiettorie Testate

---

Il controllore MPC è stato valutato su tre traiettorie rappresentative dello spazio operativo:

- traiettoria circolare (`ShapeCircle`)
- traiettoria lemniscata (di Geroni) (`ShapeLemniscate2`)
- traiettoria spirale (`ShapeSpiral`)

Per ciascuna traiettoria viene generata offline una sequenza **piecewise constant** (PWC) di reference:

$$x_{ref}(t), \quad u_{ref}(t)$$

mediante la funzione `GeneratePWC_reference.m`.

#### SUBSECTION 9.2

### Condizioni Iniziali

---

Per ogni scenario di volo vengono testate tre differenti condizioni iniziali.

#### 1. OnReference (Inizializzazione sulla Traiettoria)

- **Posizionamento del drone rispetto alla traiettoria:** il drone viene posizionato esattamente sullo stato iniziale nominale  $x_{ref}(0)$  ricavato tramite flatness map a  $t = 0$ .
- **Prima linearizzazione LTV (Stato misurato iniziale):** viene eseguita utilizzando come stato misurato lo stato completo a  $t = 0$ :

---


$$x_{0, euler} = x_{ref}(0) = [x_I(0), y_I(0), z_I(0), \dot{x}_B(0), \dot{y}_B(0), \dot{z}_B(0), \phi(0), \theta(0), \psi(0), p(0), q(0), r(0)]^T$$

$$x_{0, quat} = x_{ref}(0) = [x_I(0), y_I(0), z_I(0), \dot{x}_B(0), \dot{y}_B(0), \dot{z}_B(0), q_0(0), q_1(0), q_2(0), q_3(0), p(0), q_r(0), r(0)]^T$$


---

- **Obiettivo:** valutare la capacità del controllore di mantenere la traiettoria di volo in condizioni iniziali ideali, valutando l'impatto degli errori di discretizzazione e di linearizzazione.

## 2. Perturbation (Condizione Perturbata)

- **Posizionamento del drone rispetto alla traiettoria:** il drone presenta un offset di 0.20 m lungo gli assi  $x$  e  $y$  rispetto al punto di partenza nominale. Per isolare l'effetto dell'errore di posizione, tutte le velocità lineari nel corpo  $(\dot{x}_B, \dot{y}_B, \dot{z}_B)$ , gli angoli di roll e pitch  $(\phi, \theta)$  e le velocità angolari  $(p, q, r)$  vengono forzati a zero. La quota  $z_I$  e lo yaw  $\psi$  mantengono invece il valore nominale.
- **Prima linearizzazione LTV (Stato misurato iniziale):** viene eseguita sullo stato misurato parzialmente perturbato e azzerato:

---


$$x_{0, \text{euler}} = [x_{\text{ref},1}(0) + 0.20, y_{\text{ref},1}(0) - 0.20, z_{\text{ref},1}(0), 0, 0, 0, 0, 0, \psi_{\text{ref},1}(0), 0, 0, 0]^T$$

$$x_{0, \text{quat}} = \left[ x_{\text{ref},1}(0) + 0.20, y_{\text{ref},1}(0) - 0.20, z_{\text{ref},1}(0), 0, 0, 0, \cos\left(\frac{\psi_{\text{ref},1}(0)}{2}\right), 0, 0, \sin\left(\frac{\psi_{\text{ref},1}(0)}{2}\right), 0, 0, 0 \right]^T$$


---

- **Obiettivo:** verificare la robustezza dell'LTV-MPC nel recuperare un errore di posizionamento iniziale, analizzando i tempi di convergenza alla reference.

## 3. Origin (Decollo / Partenza dall'Origine)

- **Posizionamento del drone rispetto alla traiettoria:** il drone viene posizionato al suolo. Tutte le velocità, gli angoli di assetto e le velocità angolari sono nulli. La quota  $z_I$  iniziale è impostata a  $-0.02$  m, che simula l'altezza fisica da terra del baricentro del drone (2 cm nel sistema di riferimento NED, considerando 4 cm di altezza del frame).
- *Inizializzazione per Lemniscata:* la posizione iniziale lungo  $x_I$  viene traslata a  $+0.75$  m per allineare il drone al di sotto del lobo destro della traiettoria.
- **Prima linearizzazione LTV (Stato misurato iniziale):** viene eseguita sullo stato misurato all'origine del frame inerziale.
- *Per Cerchio e Spirale:*

---


$$x_{0, \text{euler}} = [0, 0, -0.02, 0, 0, 0, 0, 0, 0, 0, 0, 0]^T$$

$$x_{0, \text{quat}} = [0, 0, -0.02, 0, 0, 0, 1, 0, 0, 0, 0, 0]^T$$


---

– *Per Lemniscata:*

---


$$x_{0, \text{euler}} = [0.75, 0, -0.02, 0, 0, 0, 0, 0, 0, 0, 0, 0]^T$$

$$x_{0, \text{quat}} = [0.75, 0, -0.02, 0, 0, 0, 1, 0, 0, 0, 0, 0]^T$$


---

- **Obiettivo:** simulare una manovra realistica di decollo o di recupero dinamico da terra verso una traiettoria già avviata in quota, sollecitando l'LTV-MPC con ampi transitori iniziali di posizione, altezza e orientamento.

## SUBSECTION 9.3

**Metriche di Valutazione**

Le prestazioni del MPC sono valutate tramite un insieme di metriche quantitative calcolate durante la simulazione.

In particolare:

- **errore di posizione** (calcolato su  $x_I, y_I, z_I$ ):

$$e_p = |p - p_{ref}|$$

- **errore di assetto** (in gradi, calcolato su  $\phi, \theta, \psi$ ):

$$e_{\phi\theta\psi} = \left| x_{7:9} - x_{7:9}^{ref} \right|$$

- **errore sulle velocità** ( $x_B, y_B, z_B$ ).
- **errore sui rate angolari** ( $p, q, r$ ).
- **errore complessivo di stato:**

$$e_x = |x - x_{ref}| \quad \text{s.t.} \quad x \in \mathbb{R}^{12}$$

- **errore sugli input:**

$$e_u = |u - u_{ref}|$$

Vengono inoltre calcolate metriche cumulative:

- **MSE / RMSE** su posizione e stato completo.
- **RMSE Steady State** in regime stazionario (una volta raggiunta la convergenza).
- tempo di convergenza basato su threshold:

$$|e_p| < 0.05 \text{ m}$$

- statistiche (*media* e *varianza*) del **tempo di soluzione del QP**.
- numero di istanti in cui il **budget computazionale** viene superato e threshold per 80% del budget.

## SUBSECTION 9.4

**Output della Simulazione**

Per ogni combinazione di traiettoria e condizione iniziale vengono salvati:

- evoluzione degli stati e degli input.
- errori temporali e metriche cumulative.
- tempi di soluzione del QP.
- grafici di tracking.
- GIF della traiettoria (tracking 2D o 3D).
- file CSV con log completi della simulazione.

I risultati sono organizzati nella directory **results/** in sotto-cartelle dedicate per scenario e condizione iniziale, e raccolti in tabelle riassuntive globali per confronto tra i casi testati.

Per garantire la piena riproducibilità dei risultati, ogni run esporta in modo strutturato i seguenti file di log:

- **Configurazione di Sistema (`simulation_config.json`):** un file di report in formato JSON che memorizza tutti i parametri fisici del quadcopter, i vincoli e le matrici di peso dell'LTV-MPC, e i parametri geometrici delle traiettorie. Questo assicura la tracciabilità delle condizioni di simulazione.
- **Metriche Globali (Summary e Extended):** file di riepilogo che raccolgono i dati quantitativi sull'accuratezza del tracking, sull'andamento temporale degli errori di stato/input e sulle prestazioni computazionali del risolutore QP, consentendo un debug mirato e una valutazione statistica del sistema implementato.

# Risultati e Conclusioni

SECTION 10

## Risultati

---

In questa sezione vengono presentati i risultati della simulazione ottenuti impostando il tempo di campionamento a  $T_s = 0.05$  s. Le traiettorie e le condizioni iniziali proposte sono state verificate integrando il modello dinamico e il controllore MPC sviluppati, raccogliendo le metriche di valutazione precedentemente definite al fine di condurne un'analisi quantitativa.

*Remark* In questa sezione viene presentata solo una selezione di risultati a scopo dimostrativo. Una volta avviata la simulazione, è possibile analizzare tutti i grafici navigando all'interno della cartella di output dedicata.

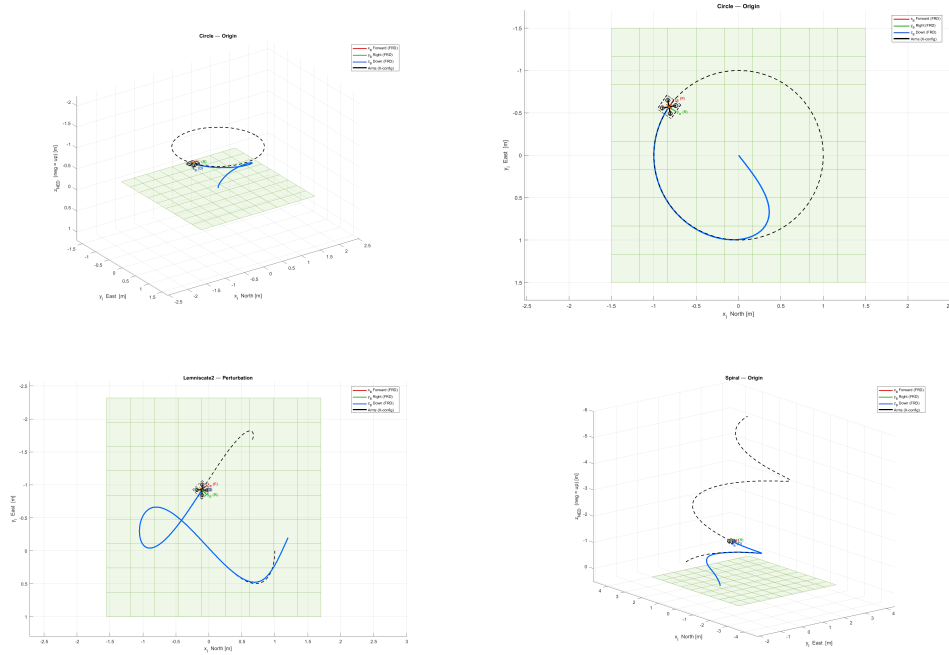
PART

V



## SUBSECTION 10.1

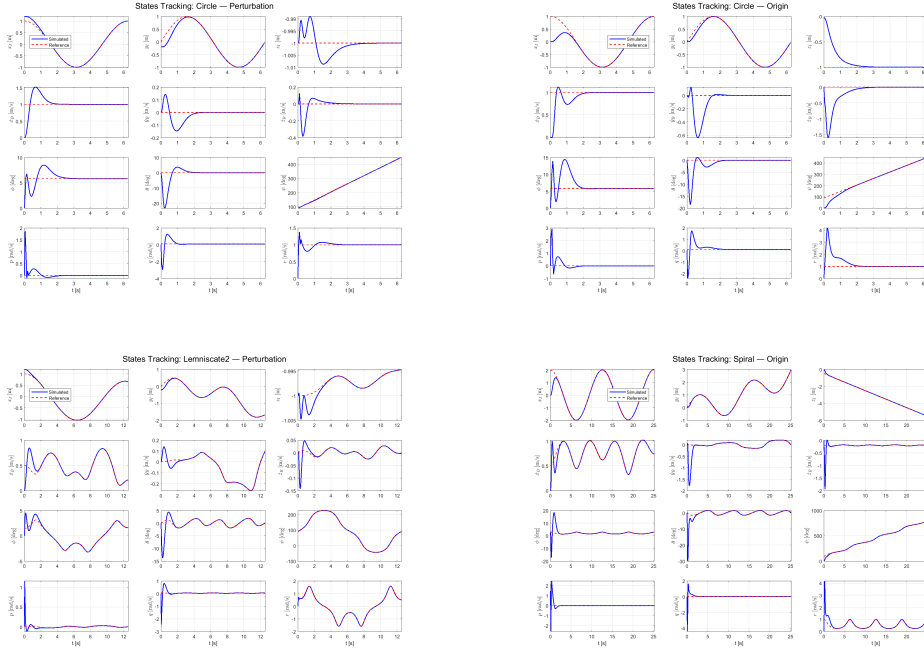
## Tracking MPC



**Figure 12.** Esempio di visualizzazione GIF del tracking MPC, con diverse condizioni iniziali e su diverse traiettorie. (a) Traiettoria circolare, visualizzazione 3D, condizione origine. (b) Traiettoria circolare, visualizzazione 2D, condizione origine. (c) Traiettoria lemniscata, visualizzazione 2D, condizione perturbata. (d) Traiettoria spirale, visualizzazione 3D, condizione origine.

## SUBSECTION 10.2

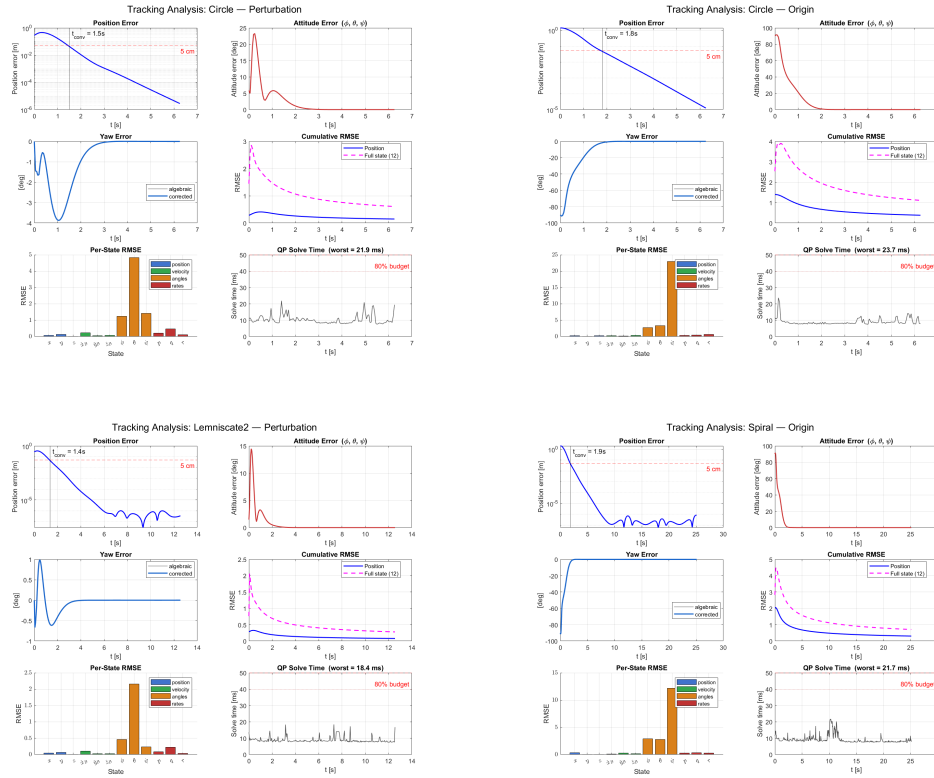
## Tracking dello Stato



**Figure 13.** Tracciamento delle variabili di stato del drone. Confronto tra la traiettoria di riferimento (linea rossa tratteggiata) e lo stato simulato del UAV controllato tramite MPC (linea blu). (a) Traiettoria circolare, condizione perturbata. (b) Traiettoria circolare, condizione origine. (c) Traiettoria lemniscata, condizione perturbata. (d) Traiettoria spirale, condizione origine.

## SUBSECTION 10.3

## Analisi del Tracking



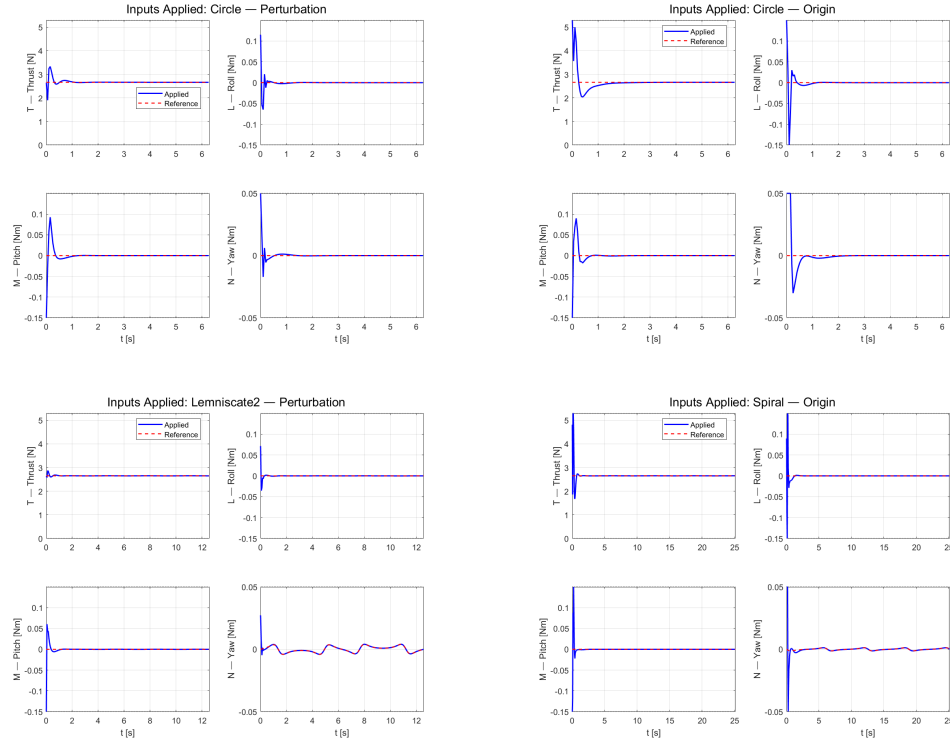
**Figure 14.** Analisi delle prestazioni di tracking in simulazione. (a) Traiettorie circolare, condizione perturbata. (b) Traiettorie circolare, condizione origine. (c) Traiettorie lemniscata, condizione perturbata. (d) Traiettorie spirale, condizione origine.

## Analisi delle prestazioni di tracking in simulazione:

- **Errore di posizione** e tempo di convergenza alla reference.
- **Errore d'assetto** (angoli di Eulero) e di imbardata (yaw).
- **RMSE cumulativo** e specifico per ogni singola variabile di stato.
- **Tempo di risoluzione** del problema QP rispetto al limite  $T_s$ .

SUBSECTION 10.4

# Tracking degli Input



**Figure 15.** Tracciamento degli input del drone. Confronto tra gli input di riferimento (linea rossa tratteggiata) e gli input applicati dal MPC al drone (linea blu). (a) Traiettoria circolare, condizione perturbata. (b) Traiettoria circolare, condizione origine. (c) Traiettoria lemniscata, condizione perturbata. (d) Traiettoria spirale, condizione origine.

SUBSECTION 10.5

# Risultati Quantitativi

## Summary

| Cond. Iniziale | Scenario   | RMSE Pos. ( $m$ )      | RMSE Stato            | RMSE Input             | QP Medio ( $ms$ ) | Steps ( $N$ ) | Durata Sim. ( $s$ ) |
|----------------|------------|------------------------|-----------------------|------------------------|-------------------|---------------|---------------------|
| OnReference    | Circle     | $5.35 \times 10^{-10}$ | $1.36 \times 10^{-8}$ | $5.82 \times 10^{-11}$ | 11.00             | 126           | 6.28                |
| OnReference    | Lemniscate | $5.63 \times 10^{-7}$  | $8.15 \times 10^{-6}$ | $1.44 \times 10^{-7}$  | 11.34             | 252           | 12.57               |
| OnReference    | Spiral     | $2.51 \times 10^{-7}$  | $1.26 \times 10^{-6}$ | $2.19 \times 10^{-8}$  | 9.44              | 503           | 25.13               |
| Perturbation   | Circle     | $1.48 \times 10^{-1}$  | $6.07 \times 10^{-1}$ | $1.17 \times 10^{-1}$  | 9.04              | 126           | 6.28                |
| Perturbation   | Lemniscate | $7.86 \times 10^{-2}$  | $2.79 \times 10^{-1}$ | $2.46 \times 10^{-2}$  | 8.55              | 252           | 12.57               |
| Perturbation   | Spiral     | $5.57 \times 10^{-2}$  | $2.04 \times 10^{-1}$ | $3.55 \times 10^{-2}$  | 8.45              | 503           | 25.13               |
| Origin         | Circle     | $3.82 \times 10^{-1}$  | $1.11 \times 10^0$    | $3.87 \times 10^{-1}$  | 9.10              | 126           | 6.28                |
| Origin         | Lemniscate | $1.84 \times 10^{-1}$  | $6.64 \times 10^{-1}$ | $2.44 \times 10^{-1}$  | 10.11             | 252           | 12.57               |
| Origin         | Spiral     | $3.05 \times 10^{-1}$  | $7.07 \times 10^{-1}$ | $2.32 \times 10^{-1}$  | 8.88              | 503           | 25.13               |

## Prestazioni di Tracciamento — Steady-State (SS)

| Cond. Iniziale | Scenario   | RMSE <sub>pos</sub> (all) [ $m$ ] | RMSE <sub>pos</sub> (SS) [ $m$ ] | RMSE <sub>stato</sub> (all) | RMSE <sub>stato</sub> (SS) | $t_{conv}$ [ $s$ ] |
|----------------|------------|-----------------------------------|----------------------------------|-----------------------------|----------------------------|--------------------|
| OnReference    | Circle     | $5.35 \times 10^{-10}$            | $5.35 \times 10^{-10}$           | $1.36 \times 10^{-8}$       | $1.36 \times 10^{-8}$      | 0.00               |
| OnReference    | Lemniscate | $5.63 \times 10^{-7}$             | $5.63 \times 10^{-7}$            | $8.15 \times 10^{-6}$       | $8.15 \times 10^{-6}$      | 0.00               |
| OnReference    | Spiral     | $2.51 \times 10^{-7}$             | $2.51 \times 10^{-7}$            | $1.26 \times 10^{-6}$       | $1.26 \times 10^{-6}$      | 0.00               |
| Perturbation   | Circle     | 0.1479                            | 0.0091                           | 0.6073                      | 0.0397                     | 1.50               |
| Perturbation   | Lemniscate | 0.0786                            | 0.0064                           | 0.2794                      | 0.0208                     | 1.35               |
| Perturbation   | Spiral     | 0.0557                            | 0.0045                           | 0.2041                      | 0.0151                     | 1.35               |
| Origin         | Circle     | 0.3824                            | 0.0122                           | 1.1102                      | 0.0382                     | 1.80               |
| Origin         | Lemniscate | 0.1839                            | 0.0080                           | 0.6644                      | 0.0190                     | 1.75               |
| Origin         | Spiral     | 0.3046                            | 0.0049                           | 0.7069                      | 0.0258                     | 1.85               |

## Errori Massimi

| Cond. Iniziale | Scenario   | Max Err. Posizione [ $m$ ] | Max Err. Yaw [ $deg$ ] | Max Err. Assetto [ $deg$ ] |
|----------------|------------|----------------------------|------------------------|----------------------------|
| OnReference    | Circle     | $8.82 \times 10^{-10}$     | $8.75 \times 10^{-7}$  | $8.79 \times 10^{-7}$      |
| OnReference    | Lemniscate | $1.19 \times 10^{-6}$      | $3.12 \times 10^{-4}$  | $3.13 \times 10^{-4}$      |
| OnReference    | Spiral     | $8.18 \times 10^{-7}$      | $8.21 \times 10^{-5}$  | $8.26 \times 10^{-5}$      |
| Perturbation   | Circle     | 0.4559                     | 3.8715                 | 23.2768                    |
| Perturbation   | Lemniscate | 0.3463                     | 0.9843                 | 14.4707                    |
| Perturbation   | Spiral     | 0.3466                     | 1.4222                 | 14.0380                    |
| Origin         | Circle     | 1.4001                     | 91.4726                | 91.5721                    |
| Origin         | Lemniscate | 1.0114                     | 90.0000                | 90.0118                    |
| Origin         | Spiral     | 2.0568                     | 91.4232                | 91.4913                    |

## Statistiche Computazionali del Solutore QP

| Cond. Iniziale | Scenario   | QP Medio [ms] | QP Peggiorre [ms] | Dev. Std. [ms] | Steps oltre l'80% $T_s$ |
|----------------|------------|---------------|-------------------|----------------|-------------------------|
| OnReference    | Circle     | 11.00         | 37.61             | 2.58           | 0                       |
| OnReference    | Lemniscate | 11.34         | 25.51             | 3.61           | 0                       |
| OnReference    | Spiral     | 9.44          | 24.04             | 2.27           | 0                       |
| Perturbation   | Circle     | 9.04          | 20.58             | 1.78           | 0                       |
| Perturbation   | Lemniscate | 8.55          | 21.48             | 1.21           | 0                       |
| Perturbation   | Spiral     | 8.45          | 19.78             | 0.97           | 0                       |
| Origin         | Circle     | 9.10          | 22.22             | 2.12           | 0                       |
| Origin         | Lemniscate | 10.11         | 20.37             | 2.40           | 0                       |
| Origin         | Spiral     | 8.88          | 23.11             | 2.02           | 0                       |

## SECTION 11

## Implementazioni Future e Conclusioni

La versione corrente costituisce una **prima validazione del processo** di simulazione LTV-MPC per il quadrotore ANT-X, e rappresenta una base di sviluppo consapevole dei propri limiti. Le criticità identificate nella sezione introduttiva comprendono:

- **Dinamica motore/ESC:** introduzione di un delay nel modello di attuazione, per rappresentare fedelmente la latenza tra comando attuato e comando fisicamente generato.
- **Stima dello stato realistica.** Sostituzione dello stato con dati rumorosi e non sincroni provenienti dalla fusione IMU + motion capture (EKF), per replicare le condizioni operative reali del laboratorio.
- **Modello aerodinamico esteso.** Aggiornamento del modello non lineare per includere *drag traslazionale* e *rotazionale*, derivate di stabilità e di controllo ricavate da El Omari [4].
- **Mixing e allocazione motori.** Implementazione della mappatura completa  $[T, L, M, N] \rightarrow [PWM_1, PWM_2, PWM_3, PWM_4]$  tramite matrice di allocazione, coefficienti di spinta  $C_T$  e coppia  $C_Q$  identificati.
- **Saturazioni di input e vincoli di stato.** I limiti sugli input e sugli stati sono stime preliminari da letteratura e richiedono verifica sperimentale. Per il modello a 13 stati (quaternione), i vincoli sulle componenti vettoriali  $[q_1, q_2, q_3]$  sono stati omessi.
- **Solver QP real-time.** Sostituzione di `quadprog` con un solver embeddable in grado di risolvere il problema di ottimizzazione entro i margini imposti dal tempo di campionamento  $T_s$ .
- **Costo terminale e stabilità.** Calcolo di  $Q_f$  come soluzione della DARE sulla linearizzazione di hover, con definizione di una regione terminale invariante per garantire stabilità asintotica.
- **Tuning sperimentale di Q e R.** Raffinamento sistematico dei pesi di costo sulla base dei risultati di volo, in sostituzione dei valori inizializzati da Kunz et al. [3].
- **Identificazione sperimentale delle inerzie.** Derivazione di  $J_{xx}$ ,  $J_{yy}$ ,  $J_{zz}$  da misure sul drone fisico (pendolo bifilare o metodo equivalente), sostituendo i valori attualmente ereditati da Cavagnini (2021) [2].

- **Nuove traiettorie e strategie di controllo.** Estensione del set di traiettorie di reference e valutazione di varianti di controllo per migliorare la robustezza e le garanzie di stabilità in scenari più aggressivi.
- **Validazione in laboratorio.** Deployment del controllore sulla piattaforma ANT-X reale (PX4 via SLXtoPX4, Vicon/OptiTrack per la stima di posizione), seguendo un protocollo incrementale: hover  $\rightarrow$  test su singoli gradi di libertà  $\rightarrow$  traiettorie complesse  $\rightarrow$  MPC.

## References

- [1] ANT-X Development Team, “ANT-X drone platform — software tools documentation,” 2024.
- [2] G. Cavagnini, “Identification and control of a nano quadrotor,” Master’s thesis, Politecnico di Milano, Dec. 2021.
- [3] L. Kunz, M. Huck, and T. H. Summers, “Fast model predictive control of miniature helicopters,” in *Proceedings of the European Control Conference (ECC)*, (Zürich, Switzerland), pp. 1377–1382, July 2013.
- [4] S. El Omari, “Multirotor UAV control via dynamic inversion,” Master’s thesis, Politecnico di Milano, 2023. A.A. 2022/2023, supervisor M. Lovera.
- [5] D. Mellinger and V. Kumar, “Minimum snap trajectory generation and control for quadrotors,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, (Shanghai, China), pp. 2520–2525, May 2011.
- [6] R. M. Murray, *Optimization-Based Control*. California Institute of Technology, 2023.
- [7] M. Faessler, A. Franchi, and D. Scaramuzza, “Differential flatness of quadrotor dynamics subject to rotor drag for accurate tracking of high-speed trajectories,” *IEEE Robotics and Automation Letters*, vol. 3, pp. 620–626, Apr. 2018.